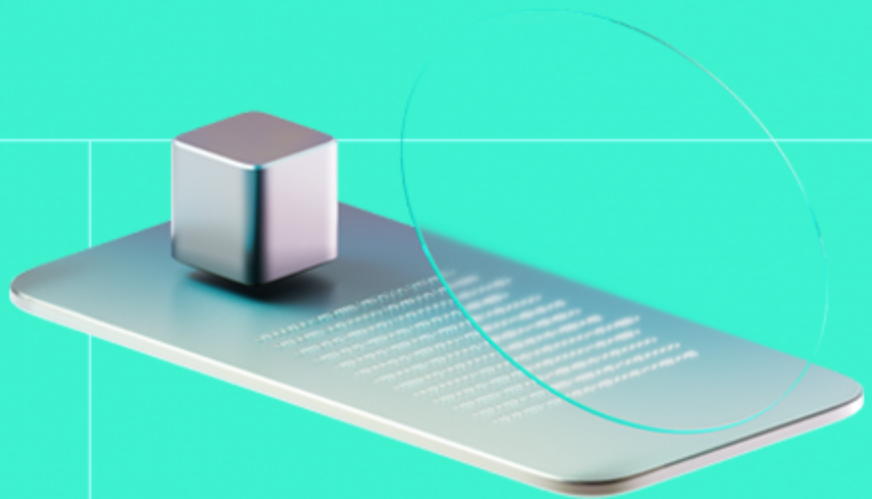




# Smart Contract Code Review And Security Analysis Report

**Customer:** Push Chain

**Date:** 05/06/2026



We express our gratitude to the Push Chain team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Push Chain is a fully EVM-compatible Proof-of-Stake Layer 1 blockchain designed to enable universal application deployment. It allows developers to deploy a Solidity-based smart contract once on Push Chain, while enabling users from multiple blockchain networks to interact with it using their existing wallets.

## Document

|             |                                                                                                                         |
|-------------|-------------------------------------------------------------------------------------------------------------------------|
| Name        | Smart Contract Code Review and Security Analysis Report for Push Chain                                                  |
| Audited By  | Olesia Bilenka; Kornel Świątłowski                                                                                      |
| Approved By | Ivan Bondar                                                                                                             |
| Website     | <a href="https://push.org">https://push.org</a>                                                                         |
| Changelog   | 31/03/2026 - Preliminary Report                                                                                         |
|             | 14/05/2026 - Remediation Report                                                                                         |
|             | 27/05/2026 - Post Audit Changes                                                                                         |
| Platform    | Push Chain, Ethereum, Solana                                                                                            |
| Language    | Solidity                                                                                                                |
| Tags        | ERC20, Signatures, Factory, Bridge                                                                                      |
| Methodology | <a href="https://docs.hacken.io/methodologies/smart-contracts">https://docs.hacken.io/methodologies/smart-contracts</a> |

## Review Scope

|                       |                                                                                                                                                             |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository            | <a href="https://github.com/pushchain/push-chain-core-contracts/tree/audit-main">https://github.com/pushchain/push-chain-core-contracts/tree/audit-main</a> |
| Commit                | 15b70544feba1586f488a90e38650bf33ea3c634                                                                                                                    |
| Remediation<br>commit | daf74825d22121896964e2d542201e623c4e994c                                                                                                                    |
| Post Audit<br>Changes | a2b61850d765679aa9e1e39998a47a348404bc08                                                                                                                    |

## Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

47

Total Findings

36

Resolved

6

Accepted

5

Mitigated

## Findings by Severity

### Severity

### Count

|          |    |
|----------|----|
| Critical | 0  |
| High     | 1  |
| Medium   | 4  |
| Low      | 14 |

| Vulnerability                                                                                                                   | Severity | Status    |
|---------------------------------------------------------------------------------------------------------------------------------|----------|-----------|
| <a href="#">F-2026-15561</a> - ERC-20 Return Path Missing Token Approval Despite Documentation Requirement                      | High     | Fixed     |
| <a href="#">F-2026-15512</a> - defaultDeadlineMins Default Value Is Not Initialized in Proxy Storage                            | Medium   | Fixed     |
| <a href="#">F-2026-15565</a> - CEA Migration Path Does Not Enforce Self-Targeting Semantics                                     | Medium   | Fixed     |
| <a href="#">F-2026-15567</a> - SVM Domain Separator Encodes chainId Incorrectly Relative to EIP-712                             | Medium   | Fixed     |
| <a href="#">F-2026-15569</a> - Domain Separator Omits Push Chain Network Identifier, Enabling Cross-Deployment Signature Replay | Medium   | Fixed     |
| <a href="#">F-2026-15501</a> - Unused isSupportedToken Gate Allows Configuration Drift                                          | Low      | Fixed     |
| <a href="#">F-2026-15504</a> - WPC.withdraw Uses .transfer (2300 Gas Limit)                                                     | Low      | Fixed     |
| <a href="#">F-2026-15507</a> - slippageTolerance Is Configurable but Never Enforced                                             | Low      | Fixed     |
| <a href="#">F-2026-15508</a> - Zero baseGasLimit Can Produce Zero Gas Fee Quote                                                 | Low      | Fixed     |
| <a href="#">F-2026-15513</a> - setDefaultFeeTier Rejects Valid Uniswap V3 1 bps Fee Tier                                        | Low      | Fixed     |
| <a href="#">F-2026-15517</a> - Deposit Event Misreports the Effective Mint Caller                                               | Low      | Fixed     |
| <a href="#">F-2026-15538</a> - Stale Gas Price Data Is Accepted Without Freshness Validation                                    | Low      | Mitigated |
| <a href="#">F-2026-15541</a> - setChainMeta Allows Zero Gas Price, Causing Quote Path DoS                                       | Low      | Fixed     |
| <a href="#">F-2026-15546</a> - Gas Token Updates Do Not Invalidate Chain Gas Price Configuration                                | Low      | Fixed     |

| Vulnerability                                                                                                             | Severity | Status    |
|---------------------------------------------------------------------------------------------------------------------------|----------|-----------|
| <a href="#">F-2026-15560</a> - Single-Call Execution Uses Implicit Dual Semantics for Value Handling                      | Low      | Mitigated |
| <a href="#">F-2026-15586</a> - VerificationType field is signed but never enforced on-chain                               | Low      | Accepted  |
| <a href="#">F-2026-15587</a> - PRC20 Direct Mint Path Bypasses UniversalCore Pause                                        | Low      | Fixed     |
| <a href="#">F-2026-15590</a> - UEA migration can be triggered by UNIVERSAL_EXECUTOR_MODULE without user signature/consent | Low      | Accepted  |
| <a href="#">F-2026-15615</a> - Absence of Granular Role Separation                                                        | Low      | Fixed     |
| <a href="#">F-2026-15505</a> - totalSupply in WPC Can Be Inflated by Force-Sent Native PC                                 | Info     | Fixed     |
| <a href="#">F-2026-15506</a> - Unused SafeERC20 Import and Inconsistent ERC-20 Handling                                   | Info     | Fixed     |
| <a href="#">F-2026-15511</a> - Missing Reentrancy Guard on depositPRC20Token                                              | Info     | Fixed     |
| <a href="#">F-2026-15519</a> - transferFrom in PRC20 Performs Balance Transfer Before Allowance Validation                | Info     | Fixed     |
| <a href="#">F-2026-15521</a> - Empty Revert Messages in WPC Reduce Failure Observability                                  | Info     | Fixed     |
| <a href="#">F-2026-15525</a> - UniversalPayload.nonce Is Ignored During Signature Hashing                                 | Info     | Mitigated |
| <a href="#">F-2026-15527</a> - Missing Events for Critical Admin Configuration Changes                                    | Info     | Fixed     |
| <a href="#">F-2026-15528</a> - PC Can Become Permanently Stuck Due to Missing Rescue Path                                 | Info     | Fixed     |
| <a href="#">F-2026-15529</a> - CEA Multicall Does Not Enforce Aggregate Value Bounds                                      | Info     | Accepted  |
| <a href="#">F-2026-15537</a> - uniswapV3Quoter Is Stored but Never Used                                                   | Info     | Fixed     |
| <a href="#">F-2026-15540</a> - Inconsistent Zero-Value Handling for Base and Rescue Gas Limits                            | Info     | Fixed     |
| <a href="#">F-2026-15542</a> - chainHeightByChainNamespace Is Stored but Unused                                           | Info     | Accepted  |
| <a href="#">F-2026-15543</a> - Role Management Surface for PAUSER_ROLE Is Inconsistent and Potentially Misleading         | Info     | Fixed     |
| <a href="#">F-2026-15544</a> - Stored Gas Pool Configuration Diverges from Runtime Swap Resolution                        | Info     | Mitigated |
| <a href="#">F-2026-15545</a> - Use of Magic Numbers Reduces Clarity and Increases Maintenance Risk in UniversalCore       | Info     | Fixed     |
| <a href="#">F-2026-15549</a> - UniversalCore Documentation Is Out of Sync With Implementation                             | Info     | Fixed     |
| <a href="#">F-2026-15550</a> - Leading-Zero Chain IDs Create Ambiguous Numeric Interpretation                             | Info     | Accepted  |
| <a href="#">F-2026-15556</a> - Return Values from IPRC20.deposit and IPRC20.approve Are Ignored                           | Info     | Fixed     |
| <a href="#">F-2026-15562</a> - SafeERC20 Is Imported but Never Used in CEA                                                | Info     | Fixed     |
| <a href="#">F-2026-15563</a> - Proxy Initialization Validation Is Inconsistent Between CEAProxy and UEAProxy              | Info     | Fixed     |
| <a href="#">F-2026-15564</a> - CEA Multicall Discards Underlying Revert Reasons                                           | Info     | Fixed     |

| Vulnerability                                                                                                  | Severity | Status    |
|----------------------------------------------------------------------------------------------------------------|----------|-----------|
| <a href="#">F-2026-15576</a> - getOriginForUEA Returns Synthetic Identity With Hardcoded Push Chain ID         | Info     | Fixed     |
| <a href="#">F-2026-15577</a> - registerUEA Can Overwrite Existing VM Implementation Mapping                    | Info     | Fixed     |
| <a href="#">F-2026-15593</a> - getOriginForUEA returns synthetic identity that may confuse authorization logic | Info     | Mitigated |
| <a href="#">F-2026-15616</a> - initializeCEA Arguments Differ From Documented Specification                    | Info     | Fixed     |
| <a href="#">F-2026-15617</a> - Gas Inefficiency Due To Dynamic String Mapping Keys                             | Info     | Accepted  |
| <a href="#">F-2026-15618</a> - Redundant Role Check in Batch Registration Increases Gas Costs                  | Info     | Fixed     |
| <a href="#">F-2026-15619</a> - Custom Errors Can Be Used for Gas Efficiency                                    | Info     | Fixed     |

## Documentation quality

- Functional requirements are provided via markdown documentation (**docs/**) covering architecture overview, UEA/CEA flows, migration mechanics, and threat modelling.
- Technical description is provided at a high level. Low-level implementation details (storage layout, upgrade invariants, exact encoding rules) are not documented.

## Code quality

- The codebase uses OpenZeppelin upgradeable contracts for access control (**AccessControlDefaultAdminRulesUpgradeable**), pausability (**PausableUpgradeable**), reentrancy guards (**ReentrancyGuardUpgradeable**), and proxy patterns (**Clones**, **ERC1967Proxy**). Custom implementations are used where protocol-specific logic is required (UEA/CEA proxy storage slots, PRC20 synthetic token, WPC wrapper).
- Shared constants and types are properly centralized in **Types.sol** and **Errors.sol**. Custom errors are used consistently across all contracts, improving gas efficiency and debuggability.
- Role-based access control follows a granular separation pattern: **ROLE\_MANAGER\_ROLE** administers operational roles, contract-specific admin roles (**UVCORE\_ADMIN\_ROLE**, **UEA\_ADMIN\_ROLE**, **CEA\_ADMIN\_ROLE**) control configuration, **OPERATOR\_ROLE** manages address setters and unpause operations, and **PAUSER\_ROLE** is restricted to pause-only actions. This design limits the blast radius of a single compromised key.
- The development environment is configured with Foundry (**forge build/forge test** toolchain), Solidity 0.8.26, Shanghai EVM target, and **via\_ir** optimization enabled.

## Test coverage

Code coverage of the project is **91.92%** (branch coverage).

- The test suite includes unit tests, fuzz tests (property-based), fork tests.
- Deployment and basic user interactions are well covered.
- Proxy initialization edge cases (**UEAProxy**, **CEAProxy**) have lower branch coverage (~50–67%).
- Cross-contract interaction scenarios (e.g., full **UEA→CEA** round-trip, migration under concurrent operations) are not tested end-to-end.

# Table of Contents

|                                         |            |
|-----------------------------------------|------------|
| <b>System Overview</b>                  | <b>8</b>   |
| Privileged Roles                        | 9          |
| <b>Potential Risks</b>                  | <b>11</b>  |
| <b>Findings</b>                         | <b>13</b>  |
| Vulnerability Details                   | 13         |
| Disclaimers                             | 112        |
| <b>Appendix 1. Definitions</b>          | <b>113</b> |
| Severities                              | 113        |
| Potential Risks                         | 113        |
| <b>Appendix 2. Scope</b>                | <b>114</b> |
| <b>Appendix 3. Additional Valuables</b> | <b>116</b> |

## System Overview

Push Chain is an EVM-compatible Proof-of-Stake Layer 1 designed for universal app execution across multiple ecosystems. Developers deploy once on Push Chain, while users from different chains interact using their native identities/wallets via protocol-managed execution accounts.

### Core Module

- **UniversalCore.sol** is the main protocol coordinator on Push Chain. It manages **PRC20** deposit/refund flows, chain gas metadata, supported gas tokens, and swap-and-burn logic through Uniswap-style integrations.
- **PRC20.sol** implements upgradeable synthetic ERC-20 assets representing external-chain tokens. It supports protocol-authorized minting (deposit), user burning, and stores source-chain token metadata plus **UniversalCore** linkage.
- **WPC.sol** is the wrapped native Push coin contract with standard wrap/unwrap and ERC-20-style transfer/allowance behavior used by core swap paths.

### UEA Module (Universal Executor Accounts)

- **UEAFactory.sol** is the upgradeable factory/registry for UEAs. It maps chain identifiers to VM types, stores UEA implementations, deploys deterministic proxy instances (**CREATE2**), and controls pause/migration configuration.
- **UEAProxy.sol** is the minimal per-account proxy that stores implementation in a fixed slot and delegates all runtime calls.
- **UEA\_EVM.sol** is the EVM **UEA** implementation. It validates ECDSA universal payloads (with trusted-module bypass path), applies nonce/deadline logic, and executes single-call, multicall, and migration dispatch.
- **UEA\_SVM.sol** is the SVM **UEA** implementation. It mirrors EVM execution flow but verifies Ed25519 signatures through the verifier precompile before dispatching calls.
- **UEAMigration.sol** is the delegatecall-only migration singleton used to update UEA proxy implementation slots (separate targets for EVM/SVM logic).

### CEA Module (Chain Executor Accounts)

- **CEAFactory.sol** is the upgradeable factory for destination-chain CEAs. It deploys deterministic proxy clones, maintains UEA↔CEA mappings, and manages vault/gateway/implementation/pause/migration parameters.
- **CEAProxy.sol** is the lightweight per-account proxy for CEAs with fixed-slot implementation storage.
- **CEA.sol** is the execution logic for CEAs. It is vault-gated, tracks executed tx IDs for replay protection, supports single-call/multicall/migration execution, and exposes outbound universal transaction initiation toward the gateway.
- **CEAMigration.sol** is the delegatecall-only migration singleton that updates CEA proxy implementation slots.

### Shared Libraries

- **Errors.sol** centralizes common and module-specific custom errors.



- **Types.sol** defines shared structs/enums/constants (universal account ID, payload formats, multical entries, selectors, and signing typehashes).
- **Utils.sol** provides utility helpers, including strict decimal-string-to-uint256 conversion (**StringUtils**).

## Privileged roles

- **UniversalCore.sol**
  - **DEFAULT\_ADMIN\_ROLE**: root governance control with 1-day timelock for admin role transfers; can grant/revoke **ROLE\_MANAGER\_ROLE**.
  - **ROLE\_MANAGER\_ROLE**: grants operational roles (**UVCORE\_ADMIN\_ROLE**, **OPERATOR\_ROLE**, **PAUSER\_ROLE**); acts as the role admin for all operational roles.
  - **UVCORE\_ADMIN\_ROLE**: controls protocol configuration — protocol fees, gas tokens, gas limits, staleness thresholds, fee tiers, auto-swap support, deadline defaults, and native PC rescue.
  - **OPERATOR\_ROLE**: address setters (**updateWPC**, **updateUniversalGatewayPC**, **updateUniswapV3Addresses**) and **unpause**.
  - **PAUSER\_ROLE**: can pause core operational entrypoints, acting as an emergency stop operator.
  - **UNIVERSAL\_EXECUTOR\_MODULE** (hardcoded: **0x14191Ea54B4c176fCf86f51b0FAc7CB1E71Df7d7**): executes module-only flows (token deposit, auto-swap, gas refund, chain metadata updates), making it a high-trust execution authority.
  - **universalGatewayPC** (privileged actor): exclusive caller for gateway burn/swap flow (**swapAndBurnGas**), so gateway integrity is a hard dependency.
- **PRC20.sol**
  - **UNIVERSAL\_EXECUTOR\_MODULE**: can update key token-level admin settings (**updateUniversalCore**, **setName**, **setSymbol**).
  - **UNIVERSAL\_CORE + UNIVERSAL\_EXECUTOR\_MODULE** (mint authority): both are allowed to mint via deposit path, so issuance trust is shared between these two actors.
- **UEAFactory.sol**
  - **DEFAULT\_ADMIN\_ROLE**: root governance with 1-day timelock; can grant/revoke **ROLE\_MANAGER\_ROLE**.
  - **ROLE\_MANAGER\_ROLE**: grants operational roles (**UEA\_ADMIN\_ROLE**, **OPERATOR\_ROLE**, **PAUSER\_ROLE**).
  - **UEA\_ADMIN\_ROLE**: controls UEA ecosystem governance — implementation mapping, migration target, chain/VM registration, Push Chain ID; defines what logic UEAs trust over time.
  - **OPERATOR\_ROLE**: can unpause UEA deployment operations.
  - **PAUSER\_ROLE**: can pause UEA deployment operations.
- **UEA\_EVM.sol and UEA\_SVM.sol**
  - **UNIVERSAL\_EXECUTOR\_MODULE**: can execute UEA payloads without user-signature verification path, including migration-triggering payload types; this is a strong delegated execution trust assumption.
- **CEAFactory.sol**
  - **DEFAULT\_ADMIN\_ROLE**: root governance with 1-day timelock; can grant/revoke **ROLE\_MANAGER\_ROLE**.
  - **ROLE\_MANAGER\_ROLE**: grants operational roles (**CEA\_ADMIN\_ROLE**, **OPERATOR\_ROLE**, **PAUSER\_ROLE**).
  - **CEA\_ADMIN\_ROLE**: controls CEA factory implementation configuration — proxy implementation, logic implementation, migration target.

- **OPERATOR\_ROLE**: address setters ( `updateVault` , `updateUniversalGateway` ) and unpause.
- **PAUSER\_ROLE**: can pause CEA deployment operations.
- **VAULT**: exclusive deploy caller for `deployCEA` , so operational control of vault affects account rollout.
- **CEA.sol**
  - **VAULT**: only inbound executor for universal tx handling; vault controls what gets executed on each CEA.
  - **address(this)** self-call gate (execution constraint): outbound gateway-send path ( `sendUniversalTxToUEA` ) is intentionally restricted to internal/self-call flows, not arbitrary external callers.

## Potential Risks

### Out-of-Scope Components & Third-Party Dependencies

- **Critical trust in privileged executor module:** A single hardcoded address embedded in bytecode across all core contracts can bypass signature checks, trigger migrations, mint synthetic tokens via privileged paths, and execute arbitrary operations on any user account without consent.
- **Bridge and Vault integrity dependency:** Chain Executor Account execution correctness depends on off-scope relay stack; any compromise there can break message authenticity and execution safety.
- **External DEX dependency for gas conversion:** **UniversalCore** relies on Uniswap V3 router and pools for fee swaps; liquidity manipulation, pool unavailability, or protocol outage can cause failed or expensive execution.
- **SVM verifier precompile dependency:** **UEA\_SVM** security and liveness depends on chain-specific Ed25519 precompile behavior and availability.

### Permissions, Authorization & Access

- **Forced migration and execution capability:** The privileged executor module path allows account execution (including migration flow) without user signature consent.
- **Admin role transfer delay:** Factory and core contracts enforce a 1-day delay on root admin role transfers. Operational role changes via the role manager remain immediate.
- **Gas oracle staleness validation (opt-in):** **UniversalCore** supports per-chain staleness checks for gas data. When configured, outbound fee queries revert if gas data exceeds the configured age. However, this check is opt-in and no hard safety bounds exist on gas price values.
- **Treasury-like rescue authority:** Protocol admin can invoke rescue functions to move protocol-held native assets.

### Centralization

- **Single hardcoded address controls all protocol-critical privileged paths:** The privileged executor module address is embedded in bytecode across **UniversalCore**, **PRC20**, **UEA\_EVM**, and **UEA\_SVM**. This single address can mint arbitrary synthetic tokens, execute any operation on any user account without signature, call privileged admin functions on token contracts, and bypass pause states on token minting. There is no enforced separation of duties, multi-signature requirement, or governance mechanism visible in the on-chain contracts. Changing this address requires a full contract redeployment or proxy implementation upgrade.
- **Factory admin controls account deployments and upgrades:** The role manager on **UEAFactory** and **CEAFactory** can grant operational admin roles, which control implementation addresses, proxy templates, migration contracts, and chain registrations. Role separation mitigates single-key compromise, but the role manager remains a single point of trust for granting operational roles.

### Upgradeability

- **Delegatecall migration trust model:** Proxy upgrade path depends on migration contracts writing storage correctly; malicious or buggy migration logic can cause systemic damage to all affected accounts.

- **Limited review window for operational changes:** While root admin role transfers have a 1-day delay, operational configuration changes via granular roles are effective immediately, reducing reaction time for users and integrators.
- **Version fragmentation:** Updates at factory level do not automatically patch already deployed account proxies; existing accounts remain on their original implementation until explicitly migrated.

## Off-Chain Dependency & Liveness

- **Liveness tied to relayer and infrastructure health:** Downtime of privileged executor module, Vault, or bridge relay components can halt cross-chain execution flows.
- **Stale configuration risk:** Delayed operational updates (e.g., gas pricing) may lead to incorrect fee behavior. The opt-in staleness check mitigates this when configured, but chains without staleness limits set remain vulnerable to stale quotes.

## Cross-Chain Consistency

- **Potential state divergence:** No strong on-chain reconciliation guarantee that Push-side intent and destination-side execution always stay synchronized.
- **Address and configuration drift over time:** Factory-level configuration changes can create operational mismatch between expected and active deployment parameters across chains.

## Remediation and Post-Audit

- **Report Modification Notice:** This report was modified on May 27th, 2026, at the client's request to incorporate minor code changes introduced up to commit `a2b6185`. While these updates were intended to align the report with the latest version of the codebase provided by the client, modifications to previously published audit content may affect the continuity and integrity of the original findings. Hacken reviewed the updated changes to confirm that they correspond solely to the requested modifications. However, any future changes involving substantial code updates or additional commits should undergo a separate security assessment to ensure that no new risks are introduced to the project's security posture.

## Findings

### Vulnerability Details

#### [F-2026-15561](#) - ERC-20 Return Path Missing Token Approval Despite Documentation Requirement - High

##### Description:

The `sendUniversalTxToUEA` function from the **CEA** provides two paths for sending tokens from a **CEA** on an external chain back to the user's **UEA** on Push Chain: a native token path and an ERC-20 token path. The native token path works correctly, forwarding `msg.value` to the gateway:

```
if (token == address(0)) {
    if (address(this).balance < amount) {
        revert CEAErrors.InsufficientBalance();
    }
    IUniversalGateway(UNIVERSAL_GATEWAY).sendUniversalTxFromCEA{value: amount}
    (req);
}
```

However, the ERC-20 path checks the token balance but never approves the gateway to pull tokens before invoking it:

```
} else {
    if (IERC20(token).balanceOf(address(this)) < amount) {
        revert CEAErrors.InsufficientBalance();
    }
    IUniversalGateway(UNIVERSAL_GATEWAY).sendUniversalTxFromCEA(req);
}
```

##### Documentation (docs/3\_CEA.md):

ERC20 path: checks balance, calls `safeApprove(UNIVERSAL_GATEWAY, amount)`, then `gateway.sendUniversalTxFromCEA(req)`.

Any gateway implementation that pulls ERC-20 tokens via `transferFrom` (which is the standard pattern, and explicitly described in the project documentation as requiring `safeApprove`) will revert with zero.

The only intended mechanism for returning ERC-20 tokens from an external chain to Push Chain (**CEA** → **UniversalGateway** → **UEA**) is non-functional for ERC-20 tokens.

Once the Vault funds a CEA with ERC-20 tokens and the user's intended action completes, any remaining ERC-20 balance cannot be returned to Push Chain through the designed flow.

**Assets:**

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:****Fixed****Classification**

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 4/5         |
| <b>Likelihood Rate:</b> | 5/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | <b>High</b> |

**Recommendations**

**Remediation:** Add `IERC20(token).approve(UNIVERSAL_GATEWAY, amount)` (or `safeApprove`) inside `sendUniversalTxToUEA` before calling the gateway for the ERC-20 path:

```
} else {  
    if (IERC20(token).balanceOf(address(this)) < amount) {  
        revert CEAEErrors.InsufficientBalance();  
    }  
    IERC20(token).safeApprove(UNIVERSAL_GATEWAY, amount);  
    IUniversalGateway(UNIVERSAL_GATEWAY).sendUniversalTxFromCEA(req);  
}
```

Consider resetting the approval to zero after the gateway call to minimize lingering allowance.

**Resolution:****Fixed in 2b70c5c :**

The CEA ERC-20 return path now approves the configured Universal Gateway before invoking `sendUniversalTxFromCEA` and resets the allowance to zero after the call. The gateway can pull ERC-20 assets via `transferFrom`, restoring the documented CEA-to-UEA ERC-20 flow

**Evidences**

## PoC: ERC20 Return Path Fails With Realistic Gateway

### Reproduce:

```
function test_PoC_ERC20ReturnPath_FailsWithRealisticGateway() public {
    ICEA cea = _deployCEAWithGateway(address(realisticGateway));

    // Fund CEA with ERC-20 tokens (simulating Vault funding)
    MockGasToken token = new MockGasToken();
    token.mint(address(cea), 1000 ether);
    assertEq(token.balanceOf(address(cea)), 1000 ether);

    // Build multicall: sendUniversalTxToUEA (ERC-20 path)
    // NOTE: No approve step – this is what the code actually does
    Multicall[] memory calls = new Multicall[](1);
    calls[0] = Multicall({
        to: address(cea),
        value: 0,
        data: abi.encodeWithSignature(
            "sendUniversalTxToUEA(address,uint256,bytes,address)",
            address(token),
            500 ether,
            "",
            ueaOnPush
        )
    });

    bytes memory payload = abi.encodePacked(MULTICALL_SELECTOR, abi.encode(calls));

    // Execute via Vault – REVERTS because gateway can't pull tokens
    vm.prank(vault);
    vm.expectRevert(Errors.ExecutionFailed.selector);
    cea.executeUniversalTx(
        keccak256("txId1"),
        keccak256("utxId1"),
        ueaOnPush,
        address(0),
        payload
    );

    // Tokens remain stuck in CEA – return path is blocked
    assertEq(token.balanceOf(address(cea)), 1000 ether, "Tokens still in CEA - return path blocked");
}
```

```
        assertEquals(realisticGateway.callCount(), 0, "Gateway was never successfully called");
    }
}
```

## Results:

```
[PASS] test_PoC_ERC20ReturnPath_FailsWithRealisticGateway() (gas: 3396048)
```



## [F-2026-15512](#) - defaultDeadlineMins Default Value Is Not Initialized in Proxy Storage - Medium

### Description:

`defaultDeadlineMins` is assigned an inline default value of `20`, but **UniversalCore** is deployed behind a proxy. For upgradeable contracts, inline state initializers are applied in the implementation contract's initialization context and are not written into proxy storage unless explicitly set during `initialize`.

```
uint256 public defaultDeadlineMins = 20;

function initialize(
    address wpc_,
    address uniswapV3Factory_,
    address uniswapV3SwapRouter_,
    address uniswapV3Quoter_,
    address initialPauser_
) public virtual initializer {
    if (initialPauser_ == address(0)) revert CommonErrors.ZeroAddress();
    __ReentrancyGuard_init();
    __AccessControl_init();
    __Pausable_init();

    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    _grantRole(PAUSER_ROLE, initialPauser_);

    emit PauserRoleGranted(initialPauser_);

    WPC = wpc_;
    uniswapV3Factory = uniswapV3Factory_;
    uniswapV3SwapRouter = uniswapV3SwapRouter_;
    uniswapV3Quoter = uniswapV3Quoter_;
}
```

The initializer never explicitly sets `defaultDeadlineMins`, so in proxy storage its value remains `0`. Therefore, the contract behavior diverges from the apparent intended default of 20 minutes.

```
if (deadline == 0) {
    deadline = block.timestamp + (defaultDeadlineMins * 1 minutes);
}
```

Any logic relying on `deadline == 0` fallback will use an effective default deadline of `block.timestamp`, not `block.timestamp + 20 minutes`.

This creates a configuration/behavior mismatch that can lead to unexpected swap execution semantics and operational confusion if deployment assumes the documented default is already active.

#### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Fixed

---

#### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 3/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

---

#### Recommendations

**Remediation:** Explicitly set the value inside `initialize`:

```
defaultDeadlineMins = 20;
```

**Resolution:** Fixed in `08d2c78`:

`defaultDeadlineMins` is no longer declared with an inline initializer and is now explicitly set to `20` inside `initialize`, ensuring the value is written into proxy storage. The proxy-side default-deadline mismatch described in the finding no longer exists.

## [F-2026-15565](#) - CEA Migration Path Does Not Enforce Self-Targeting Semantics - Medium

### Description:

The **CEA** migration flow is documented as requiring a self-targeted migration payload (`to == address(this)`), mirroring the **UEA** design. However, the actual **CEA** implementation does not validate the recipient argument at all in the migration path.

### 3\_CEA.md:

- ``MIGRATION_SELECTOR``  
(``bytes4(keccak256("UEA_MIGRATION"))``) → ``_handleMigration``:  
must be standalone (not nested in multicall). Requires ``to address(this)`` and ``value 0``. Delegatecalls  
``CEAFactory.CEA_MIGRATION_CONTRACT()`` → ``migrateCEA()``.

### CEA\_UEA\_MIGRATION\_FLOW.md:

8. `CEA._handleMigration()`:  
validates `payload.to == address(this)`  
validates `payload.value == 0`

**UEA** implementation enforces this model:

```
if (payload.to != address(this)) {  
    revert UEAErrors.InvalidCall();  
}  
  
if (payload.value != 0) {  
    revert UEAErrors.InvalidCall();  
}
```

**CEA** implementation does not:

```
} else if (_isMigration(payload)) {  
    _handleMigration();  
    emit UniversalTxExecuted(txId, universalTxId, originCaller, address(this),  
payload);  
}  
  
function _handleMigration() internal {  
    if (msg.value != 0) revert CEAErrors.InvalidInput();  
    address migrationContract = factory.CEA_MIGRATION_CONTRACT();  
    if (migrationContract == address(0)) {  
        revert CEAErrors.InvalidCall();  
    }  
  
    bytes memory migrateCallData = abi.encodeWithSignature("migrateCEA()");  
    (bool success,) = migrationContract.delegatecall(migrateCallData);
```

```
if (!success) revert CEAErrors.ExecutionFailed();  
}
```

The migration path never checks that the supplied recipient corresponds to the **CEA** itself.

The implementation does not enforce the documented self-targeting invariant for **CEA** migration. The mismatch weakens the explicitness of migration intent and makes it harder for integrators and reviewers to reason about the expected migration preconditions.

#### Assets:

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Fixed

#### Classification

|                  |                |
|------------------|----------------|
| Impact Rate:     | 4/5            |
| Likelihood Rate: | 3/5            |
| Exploitability:  | Semi-Dependent |
| Complexity:      | Simple         |
| Severity:        | Medium         |

#### Recommendations

**Remediation:** Align **CEA** migration validation with the documented and **UEA** behavior by explicitly requiring `recipient == address(this)` before executing migration logic.

**Resolution:** Fixed in `10bf9b9` :

The CEA migration path enforces the documented self-targeting invariant. `_handleExecution` passes `recipient` into `_handleMigration`, and `_handleMigration` reverts unless `recipient == address(this)` before the delegatecall.

## [F-2026-15567](#) - SVM Domain Separator Encodes chainId Incorrectly Relative to EIP-712 - Medium

**Description:** The **UEA\_SVM** contract defines an EIP-712-style domain separator with the type:

```
keccak256("EIP712Domain_SVM(string version,string chainId,address verifyingContract)")
```

This means both `version` and `chainId` are dynamic string fields and, per EIP-712 encoding rules, each must be represented in the domain separator as `keccak256(bytes(field))`.

However, `domainSeparator` from **UEA\_SVM** hashes `VERSION` correctly but passes `chainId` as a raw string into `abi.encode`:

```
function domainSeparator() public view returns (bytes32) {
    return keccak256(
        abi.encode(
            DOMAIN_SEPARATOR_TYPEHASH_SVM,
            keccak256(bytes(VERSION)),
            _universalAccountId.chainId,
            address(this)
        )
    );
}
```

As a result, the computed domain separator does not follow the standard EIP-712 encoding model for dynamic string fields. Instead of encoding `chainId` as a fixed `bytes32` hash, the contract encodes it as a dynamic string, producing a different preimage and therefore a different final domain separator.

This creates a mismatch between the contract's domain separator and the domain separator that would be produced by a standard EIP-712 implementation. As a consequence:

- standard signing libraries or off-chain tooling that implement EIP-712 correctly may derive a different domain separator than the contract;
- valid user intent may produce signatures that fail on-chain verification in **UEA\_SVM**;
- SVM integrations become dependent on custom, contract-specific hashing logic rather than standard EIP-712 behavior;
- this increases integration fragility and creates a risk of signature-validation failures across wallets, SDKs, relayers, or future tooling.

While this does not directly enable unauthorized execution, it can break the signature flow for SVM-origin accounts and cause denial of service for legitimate signed payloads.

**Assets:**

- UEA\_SVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:**

Fixed

---

**Classification**

**Impact Rate:** 3/5

**Likelihood Rate:** 3/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Medium

---

**Recommendations**

**Remediation:**

Encode `chainId` consistently with EIP-712 rules for dynamic string fields by hashing it before inclusion in the domain separator:

```
return keccak256(
    abi.encode(
        DOMAIN_SEPARATOR_TYPEHASH_SVM,
        keccak256(bytes(VERSION)),
        keccak256(bytes(_universalAccountId.chainId)),
        address(this)
    )
);
```

**Resolution:**

**Fixed in** `5d97ff4` :

The current `domainSeparator` hashes `chainId` as `keccak256(bytes(_universalAccountId.chainId))` before inclusion in `abi.encode`, aligning the encoding with EIP-712 requirements for dynamic `string` fields. Commit `5d97ff4` corrected the prior non-standard raw-string encoding, removing the domain-separator mismatch that could cause signature validation failures in standard SVM signing integrations.

## [F-2026-15569](#) - Domain Separator Omits Push Chain Network Identifier, Enabling Cross-Deployment Signature Replay - Medium

### Description:

The EIP-712 domain separator in both **UEA\_EVM** and **UEA\_SVM** is constructed using only the source chain's identifier, the contract version, and the verifying contract address. It does not include Push Chain's own network identifier (`block.chainid`).

In **UEA\_EVM**:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH =
    0x2aef22f9d7df5f9d21c56d14029233f3fdaa91917727e1eb68e504d27072d6cd;
// keccak256("EIP712Domain(string version,uint256 chainId,address verifyingContract)")
function domainSeparator() public view returns (bytes32) {
    uint256 chainId = StringUtils.stringToExactUInt256(
        _universalAccountId.chainId
    );
    return keccak256(
        abi.encode(
            DOMAIN_SEPARATOR_TYPEHASH,
            keccak256(bytes(VERSION)),
            chainId,           // source chain ID (e.g. 1 for Ethereum)
            address(this)     // UEA proxy address
                               // block.chainid - MISSING
        )
    );
}
```

In **UEA\_SVM**:

```
function domainSeparator() public view returns (bytes32) {
    return keccak256(
        abi.encode(
            DOMAIN_SEPARATOR_TYPEHASH_SVM,
            keccak256(bytes(VERSION)),
            _universalAccountId.chainId, // source chain ID
            address(this)               // UEA proxy address
                                         // block.chainid - missing
        )
    );
}
```

The `chainId` parameter in both implementations refers to the source chain (the external chain the user originates from), not the execution chain (Push

Chain). Since Push Chain can be deployed across multiple networks (testnet, mainnet, staging), and since UEA proxy addresses are deterministic via `CREATE2`, the domain separator will be identical for the same user across all Push Chain deployments.

A signature that is valid on one Push Chain deployment can be replayed on any other deployment where the same UEA exists. This enables a concrete cross-deployment replay attack:

1. **UEA** proxies are deployed via **Clones**. `cloneDeterministic` using a salt derived from `UniversalAccountId`. If the same factory and configuration exist on testnet and mainnet, the same user receives the same UEA address on both.
2. The domain separator depends only on `VERSION` (constant), source `chainId` (same user = same source chain), and `address(this)` (same via `CREATE2`). All three values are identical across deployments.
3. An attacker who observes or obtains a signed payload from a lower-value environment (testnet, staging) can replay that exact signature on mainnet where the UEA holds real assets.
4. First-time users are particularly vulnerable: their contract `nonce` is 0 on all deployments, so a signature produced on testnet with `nonce` = 0 is immediately valid on mainnet.
5. This can result in direct loss of funds - the replayed payload executes arbitrary calls (single call, multicall, or even migration) from the victim's UEA on the target deployment.

#### Assets:

- UEA\_SVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEA\_EVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 4/5         |
| Likelihood Rate: | 3/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Medium      |

#### Recommendations



## Remediation:

Include `block.chainid` in the domain separator to bind signatures to a specific Push Chain deployment:

```
bytes32 public constant DOMAIN_SEPARATOR_TYPEHASH =
    keccak256("EIP712Domain(string version,uint256 chainId,uint256 pushChainId
,address verifyingContract)");
function domainSeparator() public view returns (bytes32) {
    uint256 chainId = StringUtils.stringToExactUInt256(
        _universalAccountId.chainId
    );
    return keccak256(
        abi.encode(
            DOMAIN_SEPARATOR_TYPEHASH,
            keccak256(bytes(VERSION)),
            chainId,
            block.chainid,          // Push Chain network ID
            address(this)
        )
    );
}
```

The same change must be applied to **UEA\_SVM**. `domainSeparator`. All off-chain signing logic and SDKs must be updated to include `pushChainId` in the signed domain.

## Resolution:

**Fixed in** `62d47cc` :

Both **UEA\_EVM** and **UEA\_SVM** now include `bytes32(block.chainid)` in the EIP-712 domain separator as the canonical `salt` field, binding signatures to the executing Push Chain deployment and eliminating cross-deployment replay based on identical source-chain identifiers and deterministic proxy addresses. The current implementation no longer permits the reported replay condition across parallel Push Chain deployments.

## F-2026-15501 - Unused isSupportedToken Gate Allows Configuration

### Drift - Low

**Description:** **UniversalCore** maintains a support-list mapping `isSupportedToken` and exposes admin function `setSupportedToken`.

```
mapping(address => bool) public isSupportedToken;
mapping(address => uint256) public protocolFeeByToken;
function setSupportedToken(address prc20, bool supported) external onlyRole(MANAGER_ROLE) {
    if (prc20 == address(0)) revert CommonErrors.ZeroAddress();
    isSupportedToken[prc20] = supported;
    emit SetSupportedToken(prc20, supported);
}
```

But this flag is never enforced in execution paths (`depositPRC20Token`, `depositPRC20WithAutoSwap`, `refundUnusedGas`, `swapAndBurnGas`, or internal `_autoSwap`).

```
function _validateParams(address token, uint256 amount, address recipient) private view {
    if (token == address(0)) revert CommonErrors.ZeroAddress();
    if (recipient == address(0)) revert CommonErrors.ZeroAddress();
    if (recipient == UNIVERSAL_EXECUTOR_MODULE || recipient == address(this)) {
        revert UniversalCoreErrors.InvalidTarget();
    }
    if (amount == 0) revert CommonErrors.ZeroAmount();
}
```

Token can be marked unsupported but still processed if called through authorized flows, which creates a false sense of protection for operators/integrator and increases risk of operational misconfiguration and policy bypass at the business-logic layer.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 2/5         |
| <b>Likelihood Rate:</b> | 3/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Low         |

## Recommendations

**Remediation:** Enforce the support check in all relevant token-processing entry points, e.g.:

- `depositPRC20Token`
- `depositPRC20WithAutoSwap`
- `refundUnusedGas`
- optionally `swapAndBurnGas` for gas-token paths

Add explicit revert (e.g. `UnsupportedToken`) when `isSupportedToken[token] == false`.

If support-list is intentionally informational, remove it (and setter/event) to avoid misleading controls.

**Resolution:** **Fixed in 63c4803 :**

The unused `isSupportedToken` mapping, `setSupportedToken` setter, and related interface/event declarations have been removed from `UniversalCore`, eliminating the misleading impression that token support was enforced at runtime. The current implementation no longer exposes the non-functional control described in the finding.

## F-2026-15504 - WPC.withdraw Uses .transfer (2300 Gas Limit) - Low

### Description:

`withdraw` function in **WPC** sends native PC using Solidity `.transfer`, which forwards only 2300 gas to the recipient. This can fail for smart-contract recipients with non-trivial receive/fallback logic, making withdrawals impossible for those integrations.

```
function withdraw(uint256 wad) public {
    require(balanceOf[msg.sender] >= wad, "");
    balanceOf[msg.sender] -= wad;
    payable(msg.sender).transfer(wad);
    emit Withdrawal(msg.sender, wad);
}
```

Contract-based users (multisigs, account abstraction wallets, adapters) may be unable to withdraw. This creates integration-level DoS for recipients requiring more than 2300 gas.

### Assets:

- WPC.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 3/5         |
| Likelihood Rate: | 2/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Low         |

### Recommendations

#### Remediation:

Use low-level call instead of `.transfer`:

```
(bool ok, ) = msg.sender.call{value: wad}("");
require(ok, "TRANSFER_FAILED");
```

Because call re-enables reentrancy vectors, keep state update before external call (already done) and consider adding a reentrancy guard if

expanding logic later.

## Resolution:

**Fixed in** [4681856](#) :

`withdraw` no longer uses `.transfer` and now sends native PC with a low-level `call`, reverting with `TransferFailed` only if the transfer itself fails. The 2300-gas limitation affecting contract-based recipients is no longer present.

## F-2026-15507 - slippageTolerance Is Configurable but Never Enforced - Low

### Description:

The **UniversalCore** contract stores per-token `slippageTolerance` and exposes an admin setter, but this value is never used in swap execution logic. Actual slippage protection is only provided by caller-supplied `minPCOut`.

```
mapping(address => uint256) public slippageTolerance;
function setSlippageTolerance(address token, uint256 tolerance) external only
Admin {
    if (token == address(0)) revert CommonErrors.ZeroAddress();
    if (tolerance > 5000) {
        revert UniversalCoreErrors.InvalidSlippageTolerance();
    }
    slippageTolerance[token] = tolerance;
}
```

`_autoSwap`:

```
ISwapRouter.ExactInputSingleParams memory params = ISwapRouter.ExactInputSing
leParams({
    tokenIn: prc20,
    tokenOut: WPC,
    fee: resolvedFee,
    recipient: address(this),
    deadline: deadline,
    amountIn: amount,
    amountOutMinimum: minPCOut,
    sqrtPriceLimitX96: 0
});
pcOut = ISwapRouter(uniswapV3SwapRouter).exactInputSingle(params);
if (pcOut < minPCOut) revert UniversalCoreErrors.SlippageExceeded();
```

The contract exposes `slippageTolerance` as a configurable risk parameter, but swap execution does not reference it.

This creates a configuration–behavior inconsistency: protocol state suggests slippage controls exist, while runtime protection relies only on caller-provided `minPCOut`.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 2/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Low

## Recommendations

### Remediation:

- Either enforce `slippageTolerance[token]` in `_autoSwap` / swap flows (derive `minOut` from on-chain quote + tolerance),
- or remove this state variable and setter if slippage is intentionally caller-controlled.

### Resolution:

**Fixed in** `2add6a6` :

The unused `slippageTolerance` mapping, setter, event, and related validation constant/error were removed from **UniversalCore**. The current implementation no longer exposes a non-functional protocol slippage configuration alongside caller-controlled `minPCOut` protection.

## F-2026-15508 - Zero baseGasLimit Can Produce Zero Gas Fee Quote - Low

### Description:

`getOutboundTxGasAndFees` function falls back to `baseGasLimitByChainNamespace` when `gasLimitWithBaseLimit == 0`. If `baseLimit` is unset (`0`), the function silently keeps gas limit at `0`, causing `gasFee` to be returned as `0` instead of reverting.

```
if (gasLimitWithBaseLimit == 0) {
    gasLimitWithBaseLimit = baseLimit;
} else if (gasLimitWithBaseLimit < baseLimit) {
    revert UniversalCoreErrors.GasLimitBelowBase(gasLimitWithBaseLimit, baseLimit);
}

gasFee = gasPrice * gasLimitWithBaseLimit;
protocolFee = protocolFeeByToken[_prc20];
```

Misconfigured chains (missing base gas limit) can return zero gas fee quotes. This creates quote/runtime inconsistency and can mislead upstream components that rely on this function for fee estimation and increases operational risk during chain onboarding or config rotations.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |                |
|------------------|----------------|
| Impact Rate:     | 2/5            |
| Likelihood Rate: | 2/5            |
| Exploitability:  | Semi-Dependent |
| Complexity:      | Simple         |
| Severity:        | Low            |

### Recommendations

#### Remediation:

Add explicit validation that `baseLimit > 0` when fallback path is used, e.g.:

- revert with a dedicated error such as `ZeroBaseGasLimit`.



## Resolution:

Fixed in [9794d51](#) :

`getOutboundTxGasAndFees` now reverts with `ZeroBaseGasLimit` when `baseGasLimitByChainNamespace[chainNamespace]` is unset, instead of silently returning a zero gas fee through the fallback path. The misconfigured-chain quote inconsistency described in the finding is no longer present.

## F-2026-15513 - setDefaultFeeTier Rejects Valid Uniswap V3 1 bps Fee Tier - Low

### Description:

`setDefaultFeeTier` only allows fee tiers `500`, `3000`, and `10000`, but does not allow Uniswap V3 fee tier `100` (1 bps), which is a valid and widely used option for highly correlated pairs.

```
function setDefaultFeeTier(address token, uint24 feeTier) external onlyAdmin
{
    if (token == address(0)) revert CommonErrors.ZeroAddress();
    if (feeTier != 500 && feeTier != 3000 && feeTier != 10000) {
        revert UniversalCoreErrors.InvalidFeeTier();
    }
    defaultFeeTier[token] = feeTier;
}
```

This reduces compatibility with valid Uniswap V3 pool configuration, prevents the protocol from using 1 bps pools even when they are the most liquid or economically efficient route for a token pair and may force operators to use less optimal fee tiers or require a contract upgrade for future integration needs.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 2/5         |
| Likelihood Rate: | 2/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Low         |

### Recommendations

#### Remediation:

- Allow fee tier `100` in validation unless there is a deliberate protocol-level reason to disallow it.

- If exclusion is intentional, document that constraint explicitly in code and operational docs.

## Resolution:

### Fixed in `345b7fd`:

`setDefaultFeeTier` now accepts the Uniswap V3 `100` fee tier through the added `FEE_TIER_LOWEST` constant, alongside `500`, `3000`, and `10000`. The compatibility limitation described in the finding is no longer present.

## F-2026-15517 - Deposit Event Misreports the Effective Mint Caller - Low

### Description:

`deposit` function in the **PRC20** contract can be called by either `UNIVERSAL_CORE` or `UNIVERSAL_EXECUTOR_MODULE`, but the emitted `Deposit` event always encodes `UNIVERSAL_EXECUTOR_MODULE` as the from value, regardless of the actual caller.

```
function deposit(address to, uint256 amount) external returns (bool) {
    if (msg.sender != UNIVERSAL_CORE && msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        revert PRC20Errors.InvalidSender();
    }
    _mint(to, amount);
    emit Deposit(abi.encodePacked(UNIVERSAL_EXECUTOR_MODULE), to, amount);
    return true;
}
```

Off-chain indexers, monitors, and analytics cannot accurately determine which authorized component initiated the mint.

This creates an event/state inconsistency and reduces the forensic value of logs during incident investigation or operational review.

### Assets:

- PRC20.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 2/5         |
| Likelihood Rate: | 2/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Low         |

### Recommendations

**Remediation:**

Emit the actual caller instead of the hardcoded module address, for example:

```
emit Deposit(abi.encodePacked(msg.sender), to, amount);
```

If the event is intentionally meant to represent a conceptual source rather than the direct caller, that behavior should be explicitly documented.

**Resolution:**

**Fixed in** `d56cecc` :

`deposit` now emits `Deposit(abi.encodePacked(msg.sender), to, amount)` instead of hardcoding `UNIVERSAL_EXECUTOR_MODULE` as the source. Event logs now reflect the actual authorized caller, restoring consistency between emitted data and runtime behavior.

## F-2026-15538 - Stale Gas Price Data Is Accepted Without Freshness Validation - Low

**Description:** **UniversalCore** stores the observation timestamp for per-chain gas metadata in `timestampObservedAtByChainNamespace`.

```
mapping(string => uint256) public chainHeightByChainNamespace;
mapping(string => uint256) public timestampObservedAtByChainNamespace;
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight)
    external
    onlyUEModule
{
    gasPriceByChainNamespace[chainNamespace] = price;
    chainHeightByChainNamespace[chainNamespace] = chainHeight;
    timestampObservedAtByChainNamespace[chainNamespace] = block.timestamp;

    emit SetChainMeta(chainNamespace, price, chainHeight, block.timestamp);
}
```

But the contract never validates that the stored gas price is recent before using it in fee calculations.

```
gasPrice = gasPriceByChainNamespace[chainNamespace];
if (gasPrice == 0) revert UniversalCoreErrors.ZeroGasPrice();
gasFee = gasPrice * gasLimitWithBaseLimit;
protocolFee = protocolFeeByToken[_prc20];
```

The same issue applies to `getRescueFundsGasLimit`, which also consumes stored gas price data without checking `timestampObservedAtByChainNamespace`.

The presence of a stored timestamp suggests freshness tracking exists, but runtime logic does not enforce it.

Outdated gas price data may continue to be used indefinitely if updates stop or are delayed. This can lead to inaccurate fee quotes and operational inconsistency during oracle outages, update lag, or relayer failures.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Mitigated

## Classification

|                  |                |
|------------------|----------------|
| Impact Rate:     | 3/5            |
| Likelihood Rate: | 3/5            |
| Exploitability:  | Semi-Dependent |
| Complexity:      | Simple         |
| Severity:        | Low            |

---

## Recommendations

- Remediation:** Introduce a freshness bound, for example:
- revert if `block.timestamp - timestampObservedAtByChainNamespace[chainNamespace] > MAX_STALENESS`, or
  - expose and document that gas metadata is informational only and freshness must be enforced off-chain.

**Resolution:** **Mitigated in `bb2178a`:**

**UniversalCore** now includes an opt-in per-chain freshness guard via `maxStalenessByChainNamespace`, and both `getOutboundTxGasAndFees` and `getRescueFundsGasLimit` call `_validateGasDataFreshness` to revert with `StaleGasData` when configured data is older than the permitted age. The underlying risk is reduced but not fully eliminated, because the protection is disabled by default when `maxStalenessByChainNamespace[chainNamespace] == 0`.

## F-2026-15541 - setChainMeta Allows Zero Gas Price, Causing Quote Path DoS - Low

### Description:

`setChainMeta` function allows the privileged caller to set a chain gas price to `0`. Once stored, both public gas-quote functions revert because they explicitly reject zero gas price values.

```
function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight)
    external
    onlyUEModule
{
    gasPriceByChainNamespace[chainNamespace] = price;
    chainHeightByChainNamespace[chainNamespace] = chainHeight;
    timestampObservedAtByChainNamespace[chainNamespace] = block.timestamp;
    emit SetChainMeta(chainNamespace, price, chainHeight, block.timestamp);
}
```

`getOutboundTxGasAndFees`:

```
gasPrice = gasPriceByChainNamespace[chainNamespace];
if (gasPrice == 0) revert UniversalCoreErrors.ZeroGasPrice();
```

`getRescueFundsGasLimit`:

```
gasPrice = gasPriceByChainNamespace[chainNamespace];
if (gasPrice == 0) revert UniversalCoreErrors.ZeroGasPrice();
```

A zero-price update disables `getOutboundTxGasAndFees` and `getRescueFundsGasLimit` for the affected chain. This creates an avoidable availability risk in fee-quote flows due to misconfiguration or bad oracle input.

Because `setChainMeta` stores the value without validation, the contract accepts invalid state that later breaks dependent functionality.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification



|                         |           |
|-------------------------|-----------|
| <b>Impact Rate:</b>     | 3/5       |
| <b>Likelihood Rate:</b> | 2/5       |
| <b>Exploitability:</b>  | Dependent |
| <b>Complexity:</b>      | Simple    |
| <b>Severity:</b>        | Low       |

---

## Recommendations

**Remediation:** Validate `price > 0` in `setChainMeta` before writing state. If a zero-price sentinel is intentionally needed, quote paths should handle it explicitly rather than reverting generically.

**Resolution:** **Fixed in `0e3af5c` :**

`setChainMeta` now rejects zero gas prices instead of storing them, preventing invalid state that would later break `getOutboundTxGasAndFees` and `getRescueFundsGasLimit`. The zero-price misconfiguration path described in the finding is no longer present.

## [F-2026-15546](#) - Gas Token Updates Do Not Invalidate Chain Gas Price Configuration - Low

### Description:

`setGasTokenPRC20` allows changing the gas token for a chain namespace, but does not reset or enforce reconfiguration of `gasPriceByChainNamespace` for that same namespace. Since fee calculation depends on both values, stale gas-price data may remain active after a token switch.

```
function setGasTokenPRC20(string memory chainNamespace, address prc20) external
onlyRole(MANAGER_ROLE) {
    if (prc20 == address(0)) revert CommonErrors.ZeroAddress();
    gasTokenPRC20ByChainNamespace[chainNamespace] = prc20;
    emit SetGasToken(chainNamespace, prc20);
}
```

`getOutboundTxGasAndFees`:

```
function getOutboundTxGasAndFees(address _prc20, uint256 gasLimitWithBaseLimit)
public view
returns(address gasToken, uint256 gasFee, uint256 protocolFee, uint256 gasPrice,
        string memory chainNamespace) {
    chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();
    uint256 baseLimit = baseGasLimitByChainNamespace[chainNamespace];

    if (gasLimitWithBaseLimit == 0) {
        gasLimitWithBaseLimit = baseLimit;
    } else if (gasLimitWithBaseLimit < baseLimit) {
        revert UniversalCoreErrors.GasLimitBelowBase(gasLimitWithBaseLimit, baseLimit);
    }

    gasToken = gasTokenPRC20ByChainNamespace[chainNamespace];
    if (gasToken == address(0)) revert CommonErrors.ZeroAddress();

    gasPrice = gasPriceByChainNamespace[chainNamespace];
    if (gasPrice == 0) revert UniversalCoreErrors.ZeroGasPrice();

    gasFee = gasPrice * gasLimitWithBaseLimit;
    protocolFee = protocolFeeByToken[_prc20];
}
```

After changing the gas token, the existing chain gas price may no longer represent the correct pricing context for the new token.

This can produce incorrect fee quotes if operators forget to update gas price immediately after token changes.

**Assets:**

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:****Fixed****Classification****Impact Rate:**

3/5

**Likelihood Rate:**

2/5

**Exploitability:**

Semi-Dependent

**Complexity:**

Simple

**Severity:****Low****Recommendations****Remediation:**

Enforce consistency on token change, for example by:

- resetting `gasPriceByChainNamespace[chainNamespace]` to `0` when gas token is changed (forcing explicit reconfiguration), or
- introducing an atomic setter that updates gas token and gas price together.

**Resolution:****Fixed in `4f2de47` :**

`setGasTokenPRC20` now resets `gasPriceByChainNamespace[chainNamespace]` to `0` whenever the gas token is changed, forcing explicit reconfiguration through `setChainMeta` before quotes can resume. The stale-price-after-token-switch condition described in the finding is no longer present.

## F-2026-15560 - Single-Call Execution Uses Implicit Dual Semantics for Value Handling - Low

### Description:

The single-call execution path intentionally supports two different behaviors depending solely on whether payload is empty:

- `payload.length == 0` → funds are parked on the CEA
- `payload.length != 0` → the entire `msg.value` is forwarded to recipient

This behavior is documented in code comments, so it does not appear to be an implementation bug. However, it creates an API surface where materially different value flows are selected implicitly by payload emptiness rather than by an explicit execution mode.

```
/// @dev Handles single-call execution or funds parking.
///      Empty payload = park funds. Non-empty = execute call.
///      Self-calls blocked (use multicall path instead).
if (payload.length == 0) {
    emit UniversalTxExecuted(txId, universalTxId, originCaller, address(this),
payload);
    return;
}
if (recipient == address(0)) {
    revert CEAErrors.InvalidRecipient();
}
if (recipient == address(this)) {
    revert CEAErrors.InvalidRecipient();
}
(bool success,) = recipient.call{value: msg.value}(payload);
if (!success) revert CEAErrors.ExecutionFailed();
```

The same entrypoint can either retain funds on the CEA or forward all attached value externally, depending only on an implicit payload condition.

This increases the chance of operator or integration mistakes in Vault-driven execution flows.

Misconstructed requests may still succeed, but with unintended value-routing behavior.

### Assets:

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Mitigated

## Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 2/5         |
| <b>Likelihood Rate:</b> | 2/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Low         |

---

## Recommendations

**Remediation:** Consider making these two behaviors explicit, for example by:

- separating them into distinct execution modes or functions, or
- emitting a dedicated event for the “park funds” path and documenting the behavior prominently in external docs.

**Resolution:** **Mitigated in 4f57902 :**

The single-call endpoint still supports both funds parking and external execution, so the unified dual-purpose API surface remains. Commit 4f57902 reduced the risk by requiring an explicit zero `recipient` in addition to an empty `payload` for funds-parking mode, removing the earlier ambiguity where payload emptiness alone determined whether assets were retained on the CEA.

## F-2026-15586 - VerificationType field is signed but never enforced on-chain - Low

**Description:** `UniversalPayload` includes `vType` (`signedVerification` / `universalTxVerification`) and this field is part of the signed payload hash (EIP-712 path).

```
struct UniversalPayload {
    address to; // Target contract address to call
    uint256 value; // Native token amount to send
    bytes data; // Call data for the function execution
    uint256 gasLimit; // Max gas for this tx (caps refund amount)
    uint256 maxFeePerGas; // Maximum fee per gas unit
    uint256 maxPriorityFeePerGas; // Maximum priority fee per gas unit
    uint256 nonce; // Nonce of the transaction
    uint256 deadline; // Timestamp after which payload is invalid
    VerificationType vType; // Verification type before execution
}

enum VerificationType {
    signedVerification, // Standard signature verification
    universalTxVerification // Universal Executor Module (no signature)
}
```

However, execution logic does not enforce behavior based on this field.

In both UEA implementations, runtime authorization is decided by caller context (`msg.sender == UNIVERSAL_EXECUTOR_MODULE`), while `vType` itself is not validated against that context and does not affect control flow.

`vType` is cryptographically committed but semantically ignored.

### Assets:

- Types.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Accepted

### Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Low

---

## Recommendations

**Remediation:** Choose one of the following and keep code/docs aligned:

- Enforce `vType` on-chain
  - If caller is `UNIVERSAL_EXECUTOR_MODULE`, require `vType == universalTxVerification`.
  - If caller is not module, require `vType == signedVerification`.
- Remove `vType` from signed struct if no behavioral branching is intended.

**Resolution:**

**Accepted:**

`vType` remains part of the signed `UniversalPayload` hash, but the current **UEA\_EVM** and **UEA\_SVM** execution paths do not enforce any runtime behavior based on that field. Authorization is still determined exclusively by caller context (`msg.sender == UNIVERSAL_EXECUTOR_MODULE` versus signature verification), and no remediation is present that validates `vType` against that context or uses it in control flow.

## F-2026-15587 - PRC20 Direct Mint Path Bypasses UniversalCore

### Pause - Low

**Description:** `deposit` function from **PRC20** contract allows minting when the caller is either:

- `UNIVERSAL_CORE`, or
- `UNIVERSAL_EXECUTOR_MODULE`

There is no pause-state check inside **PRC20**:

```
function deposit(address to, uint256 amount) external returns (bool) {
    if (msg.sender != UNIVERSAL_CORE && msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        revert PRC20Errors.InvalidSender();
    }
    _mint(to, amount);
}
```

So even if **UniversalCore** is paused, minting can still proceed through the direct privileged path (`UNIVERSAL_EXECUTOR_MODULE` → **PRC20**.`deposit`).

If operations teams/users interpret `pause` function from **UniversalCore** as a global emergency stop for synthetic token issuance, this assumption is `false`.

Minting remains available via the direct executor path, weakening pause guarantees and incident-response expectations.

### Assets:

- PRC20.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 2/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Low



## Recommendations

**Remediation:** Consider adding pause gating to `deposit` (or query core pause state) so minting halts during protocol pause, including executor path.

**Resolution:** **Fixed in `d9a0f6e` :**

`PRC20.deposit()` now checks `PausableUpgradeable(UNIVERSAL_CORE).paused()` and reverts with `CorePaused` before minting, including calls from `UNIVERSAL_EXECUTOR_MODULE`. The direct privileged mint path no longer bypasses the `UniversalCore` pause state.

## [F-2026-15590](#) - UEA migration can be triggered by UNIVERSAL\_EXECUTOR\_MODULE without user signature/consent - Low

### Description:

Both **UEA** implementations skip signature verification when caller is `UNIVERSAL_EXECUTOR_MODULE`.

Because migration is just one payload mode handled inside the same execution path, the module can trigger migration without any user signature.

```
function executeUniversalTx(
    UniversalPayload calldata payload,
    bytes calldata signature
) external nonReentrant {
    if (msg.sender != UNIVERSAL_EXECUTOR_MODULE) {
        bytes32 payloadHash = getUniversalPayloadHash(payload);
        if (!verifyUniversalPayloadSignature(payloadHash, signature)) {
            revert UEAErrors.InvalidEVMSignature();
        }
    }
    _handleExecution(payload);
}
```

`_handleExecution`:

```
if (_isMulticall(payload.data)) {
    (success, returnData) = _handleMulticall(payload);
} else if (_isMigration(payload.data)) {
    (success, returnData) = _handleMigration(payload);
} else {
    (success, returnData) = _handleSingleCall(payload);
}
```

`_handleMigration`:

```
address migrationContract = ueaFactory.UEA_MIGRATION_CONTRACT();
bytes memory migrateCallData = abi.encodeWithSignature("migrateUEAEVM()");
(success, returnData) = migrationContract.delegatecall(migrateCallData);
```

Same logic exists in **UEA\_SVM**.

- User consent is not required for migration timing/execution once module calls the entrypoint.

- Operational trust in `UNIVERSAL_EXECUTOR_MODULE` is stronger than “relayer” semantics; it effectively has upgrade-trigger power over all UEAs.
- If this trust assumption is not explicit in docs/governance, users/integrators may assume migration needs user authorization when it does not.

**Assets:**

- UEA\_SVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEA\_EVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:**`Accepted`**Classification****Impact Rate:** 4/5**Likelihood Rate:** 2/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** `Low`**Recommendations**

**Remediation:** Consider requiring signature verification for migration payloads even when caller is the module.

**Resolution:****Accepted:**

The migration path through `UNIVERSAL_EXECUTOR_MODULE` reflects an explicit protocol-trust assumption equivalent to trusting Push Chain's consensus-controlled inbound dispatch mechanism. Direct callers outside that path still require user signatures, so no separate contract-level attack vector remains once the module account is treated as a trusted protocol actor. Documentation should state clearly that module-triggered migration does not require per-user authorization and inherits the trust assumptions of the chain and validator consensus.

## F-2026-15615 - Absence of Granular Role Separation - Low

### Description:

The **UEAFactory** , **UniversalCore**, **CEAFactory** contracts implement the **DEFAULT\_ADMIN\_ROLE** , which currently grants extensive privileges across multiple administrative operations, including setter functions. The wide scope of authority assigned to this role introduces a security concern when it is used for day-to-day administrative tasks.

A compromise of the private key controlling this role could result in full unauthorized access to key protocol functions. This may lead to unauthorized configuration changes, disruption of system behavior, or full control over protocol parameters. The current design lacks role segregation, where distinct operational and administrative responsibilities are assigned to separate roles with limited scopes of authority.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- CEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

**Impact Rate:** 4/5

**Likelihood Rate:** 2/5

**Exploitability:** Dependent

**Complexity:** Simple

**Severity:** Low

### Recommendations

#### Remediation:

It is recommended to introduce granular role segregation by creating separate role like **ADMIN\_ROLE** for routine administrative operations, higher-privilege management functions, and **UPGRADER\_ROLE** for contract upgrades. The **DEFAULT\_ADMIN\_ROLE** should be reserved exclusively for roles management.

## Resolution:

### Fixed in 6215a51 :

Administrative privileges have been separated from `DEFAULT_ADMIN_ROLE` through **AccessControlDefaultAdminRulesUpgradeable** and dedicated operational roles. **UEAFactory**, **UniversalCore**, and **CEAFactory** now define role-specific authorities such as `ROLE_MANAGER_ROLE`, contract-specific admin roles, `OPERATOR_ROLE`, and `PAUSER_ROLE`. Setter and operational functions are protected by scoped roles rather than direct `DEFAULT_ADMIN_ROLE` checks, reducing day-to-day reliance on the root admin role.

## [F-2026-15505](#) - totalSupply in WPC Can Be Inflated by Force-Sent Native PC - Info

### Description:

`totalSupply` function of **WPC** contract is defined as the contract's native balance (`address(this).balance`). This can be increased without calling `deposit` by forcibly sending native PC (e.g., via `selfdestruct` from another contract). In that case, native balance increases but `balanceOf` mappings do not, breaking expected ERC-20 accounting parity.

```
function totalSupply() public view returns (uint256) {  
    return address(this).balance;  
}
```

`totalSupply` may become greater than the sum of all user balances. Off-chain indexers/integrators assuming strict ERC-20 invariants can observe inconsistent state.

### Assets:

- WPC.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

## Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 2/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

## Recommendations

### Remediation:

Track supply in storage (`_totalSupply`) and update only in `deposit` / `withdraw`, instead of using `address(this).balance`.

### Resolution:

Fixed in [753394d](#) :

`totalSupply` now returns a dedicated `_totalSupply` storage variable that is updated only in `deposit` and `withdraw`, rather than `address(this).balance`.

Forced native transfers no longer inflate reported token supply or break ERC-20 accounting parity.

## F-2026-15506 - Unused SafeERC20 Import and Inconsistent ERC-20 Handling - Info

### Description:

**SafeERC20** is imported and attached via `using SafeERC20 for IERC20`, but token approvals are performed with raw `approve(...)` calls instead of safe wrappers. This creates inconsistency in coding style and weakens defensive handling for non-standard ERC-20 tokens.

```
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

```
using SafeERC20 for IERC20;
```

`swapAndBurnGas` :

```
IERC20(WPC).approve(uniswapV3SwapRouter, msg.value);
IERC20(WPC).approve(uniswapV3SwapRouter, 0);
```

`_autoSwap` :

```
IPRC20(prc20).approve(uniswapV3SwapRouter, amount);
IPRC20(prc20).approve(uniswapV3SwapRouter, 0);
```

This reduces robustness if token behavior changes in future integrations and adds dead/unused abstraction and can mislead reviewers about actual safety guarantees.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple



**Severity:**

Info

**Recommendations****Remediation:**

- Either switch to **SafeERC20** methods consistently (`safeApprove` /preferred allowance pattern), or
- remove the unused **SafeERC20** import/using statement if intentionally not needed.

**Resolution:****Fixed in `dc72213` :**

Raw `approve` calls in `swapAndBurnGas` and `_autoSwap` have been replaced with `SafeERC20.forceApprove`, and the approval-reset paths were updated accordingly. The current implementation now uses the safe wrapper consistently for router approvals, removing the inconsistency described in the finding.

## [F-2026-15511](#) - Missing Reentrancy Guard on depositPRC20Token - Info

### Description:

`depositPRC20Token` function performs an external call to a token contract (`IPRC20(prc20).deposit(...)`) but is the only UE-module state-changing entrypoint that is not protected by `nonReentrant`. Other adjacent execution paths (`depositPRC20WithAutoSwap`, `refundUnusedGas`, `swapAndBurnGas`) do use reentrancy protection.

```
function depositPRC20Token(address prc20, uint256 amount, address recipient)
external onlyUEModule whenNotPaused {
    _validateParams(prc20, amount, recipient);
    IPRC20(prc20).deposit(recipient, amount);
}
```

This creates a security-hardening inconsistency across similar privileged flows.

If a non-standard or upgraded token implementation introduces callback-capable behavior, this path may become reentrancy-reachable before defenses are added.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 3/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

### Recommendations

#### Remediation:

Add `nonReentrant` to `depositPRC20Token` for symmetry and defense-in-depth, unless there is a strict gas/design reason not to.

**Resolution:****Fixed in `d41ba00` :**

`depositPRC20Token` now includes the `nonReentrant` modifier, aligning its protection with the adjacent UE-module entrypoints that already used reentrancy guards. The hardening inconsistency described in the finding is no longer present.

## F-2026-15519 - transferFrom in PRC20 Performs Balance Transfer Before Allowance Validation - Info

### Description:

`transferFrom` function in **PRC20** contract calls `_transfer` before checking whether `msg.sender` has sufficient allowance from sender. This inverts the standard ERC-20 execution order, where allowance is normally validated and reduced before balances are moved.

```
function transferFrom(address sender, address recipient, uint256 amount) external returns (bool) {
    _transfer(sender, recipient, amount);
    uint256 currentAllowance = _allowances[sender][msg.sender];
    if (currentAllowance < amount) revert PRC20Errors.LowAllowance();
    unchecked {
        _allowances[sender][msg.sender] = currentAllowance - amount;
    }
    emit Approval(sender, msg.sender, _allowances[sender][msg.sender]);
    return true;
}
```

The implementation deviates from expected ERC-20 control flow and violates the usual checks-before-effects pattern.

Although a failing allowance check will revert the entire transaction under current Solidity semantics, the ordering still creates unnecessary semantic risk and makes the code harder to reason about.

If transfer logic is ever extended with hooks, callbacks, or external interactions in a future version, this ordering can become materially dangerous.

### Assets:

- PRC20.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

**Severity:**

Info

**Recommendations****Remediation:**

Validate and deduct allowance before calling `_transfer`, following standard ERC-20 behavior:

1. Read current allowance
2. Revert if insufficient
3. Decrease allowance
4. Execute token transfer

**Resolution:****Fixed in `b98313f`:**

`PRC20.transferFrom` now validates and deducts allowance before calling `_transfer`, restoring the standard ERC-20 execution order and removing the checks-after-effects inconsistency described in the finding.

## F-2026-15521 - Empty Revert Messages in WPC Reduce Failure Observability - Info

**Description:** Several require statements in **WPC** contract use empty revert strings `("")`. While this does not directly affect correctness, it makes failures harder to diagnose for integrators, testers, monitoring systems, and users.

```
function withdraw(uint256 wad) public {
    require(balanceOf[msg.sender] >= wad, "");
    balanceOf[msg.sender] -= wad;
    payable(msg.sender).transfer(wad);
    emit Withdrawal(msg.sender, wad);
}

function transferFrom(address src, address dst, uint256 wad) public returns (
    bool) {
    require(balanceOf[src] >= wad, "");
    if (src != msg.sender && allowance[src][msg.sender] != type(uint256).max)
    {
        require(allowance[src][msg.sender] >= wad, "");
        ...
    }
}
```

Reverted transactions provide no meaningful reason string, making off-chain troubleshooting and operational monitoring harder.

### Assets:

- WPC.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 1/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

## Recommendations

### Remediation:

Replace empty revert messages with:

- descriptive revert strings, or preferably
- custom errors for lower gas cost and better structured failure reporting.

### Resolution:

**Fixed in** `31d5417` :

Empty revert strings in `WPC` have been replaced with explicit custom errors, including `WPCErrors.InsufficientBalance` and `WPCErrors.InsufficientAllowance`. The current implementation provides structured failure reasons for the affected `withdraw` and `transferFrom` paths.

## [F-2026-15525](#) - UniversalPayload.nonce Is Ignored During Signature Hashing - Info

### Description:

The `UniversalPayload` struct includes a nonce field, which implies caller-controlled transaction ordering and explicit nonce binding in the signed payload.

```
struct UniversalPayload {
    address to; // Target contract address to call
    uint256 value; // Native token amount to send
    bytes data; // Call data for the function execution
    uint256 gasLimit; // Max gas for this tx (caps refund amount)
    uint256 maxFeePerGas; // Maximum fee per gas unit
    uint256 maxPriorityFeePerGas; // Maximum priority fee per gas unit
    uint256 nonce; // Nonce of the transaction
    uint256 deadline; // Timestamp after which payload is invalid
    VerificationType vType; // Verification type before execution
}
```

However, both UEA implementations ignore `payload.nonce` and instead use the contract's current storage nonce when computing the signed hash.

### UEA\_EVM:

```
function getUniversalPayloadHash(
    UniversalPayload memory payload
) public view returns (bytes32) {
    bytes32 structHash = keccak256(
        abi.encode(
            UNIVERSAL_PAYLOAD_TYPEHASH,
            payload.to,
            payload.value,
            keccak256(payload.data),
            payload.gasLimit,
            payload.maxFeePerGas,
            payload.maxPriorityFeePerGas,
            nonce,
            payload.deadline,
            uint8(payload.vType)
        )
    );
}
```



## UEA\_SVM:

```
function getUniversalPayloadHash(
    UniversalPayload memory payload
) public view returns (bytes32) {
    bytes32 structHash = keccak256(
        abi.encode(
            UNIVERSAL_PAYLOAD_TYPEHASH,
            payload.to,
            payload.value,
            keccak256(payload.data),
            payload.gasLimit,
            payload.maxFeePerGas,
            payload.maxPriorityFeePerGas,
            nonce,
            payload.deadline,
            uint8(payload.vType)
        )
    );
}
```

The payload structure suggests that users can explicitly bind and reason about transaction ordering via `payload.nonce`, but this guarantee is not actually enforced. Signers, SDKs, and integrators may incorrectly assume that `payload.nonce` is part of the signed intent, creating a mismatch between signed data semantics and runtime behavior.

This breaks the expected **EIP-712** trust model: the signer cannot reliably reason about the exact nonce being authorized from the payload itself. In practice, integrations may sign one nonce while the contract verifies another.

The problem is amplified by the protocol's cross-chain design:

- users sign on an origin chain or in an off-chain flow, while nonce state lives on Push Chain;
- relay latency and concurrent execution make live nonce synchronization fragile;
- the payload format explicitly advertises signer-controlled ordering, but that ordering is not actually enforced by the signed data.

### Assets:

- UEA\_SVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEA\_EVM.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Mitigated

## Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 3/5         |
| <b>Likelihood Rate:</b> | 1/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

---

## Recommendations

**Remediation:** Choose one model and enforce it consistently:

- require `payload.nonce == nonce` and hash `payload.nonce`, or
- remove the nonce field from `UniversalPayload` entirely if nonce is intended to be implicit contract state.

Also update documentation to clearly describe the actual nonce model used for signing and replay protection.

**Resolution:** **Mitigated in `daf7482`:**

The `executeUniversalTx` function in both `UEA_EVM` and `UEA_SVM` now enforces an explicit check that `payload.nonce == nonce` before execution proceeds. A new `NonceMismatch(uint256 expected, uint256 provided)` error is emitted when the provided nonce does not match the contract's storage nonce.

The hash computation in `getUniversalPayloadHash` still uses the storage `nonce` rather than `payload.nonce`. However, because the explicit equality check requires `payload.nonce` to match the storage `nonce`, the signed data and runtime state are now consistent at execution time. Signers must provide the correct current nonce in the payload; any mismatch causes immediate reversion before signature verification.

## [F-2026-15527](#) - Missing Events for Critical Admin Configuration

### Changes - Info

#### Description:

Several privileged configuration functions update important protocol state without emitting dedicated events. This reduces transparency for off-chain monitoring, makes governance actions harder to audit, and weakens operational visibility during incident response or configuration reviews.

In **UniversalCore**, multiple admin-only setters mutate critical swap and access-control configuration without events:

```
function setAutoSwapSupported(address token, bool supported) external onlyAdmin
in {
    isAutoSwapSupported[token] = supported;
}

function setWPC(address addr) external onlyAdmin {
    if (addr == address(0)) revert CommonErrors.ZeroAddress();
    WPC = addr;
}

function setUniversalGatewayPC(address addr) external onlyAdmin {
    if (addr == address(0)) revert CommonErrors.ZeroAddress();
    universalGatewayPC = addr;
}

function setUniswapV3Addresses(address factory, address swapRouter, address q
uoter) external onlyAdmin {
    if (factory == address(0) || swapRouter == address(0) || quoter == address
(0)) {
        revert CommonErrors.ZeroAddress();
    }
    uniswapV3Factory = factory;
    uniswapV3SwapRouter = swapRouter;
    uniswapV3Quoter = quoter;
}

function setDefaultFeeTier(address token, uint24 feeTier) external onlyAdmin
{
    if (token == address(0)) revert CommonErrors.ZeroAddress();
    if (feeTier != 500 && feeTier != 3000 && feeTier != 10000) {
        revert UniversalCoreErrors.InvalidFeeTier();
    }
    defaultFeeTier[token] = feeTier;
}
```

```
function setSlippageTolerance(address token, uint256 tolerance) external only
Admin {
    if (token == address(0)) revert CommonErrors.ZeroAddress();
    if (tolerance > 5000) {
        revert UniversalCoreErrors.InvalidSlippageTolerance();
    }
    slippageTolerance[token] = tolerance;
}
```

In **PRC20**, metadata-changing admin functions also do not emit events:

```
function setName(string memory newName) external onlyUniversalExecutor {
    _name = newName;
}
function setSymbol(string memory newSymbol) external onlyUniversalExecutor {
    _symbol = newSymbol;
}
```

Off-chain observers cannot reliably detect or reconstruct important configuration changes from logs alone. Governance and security monitoring become less robust, especially for changes affecting routing, execution permissions, token behavior, or metadata.

#### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- PRC20.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Fixed

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 2/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

#### Recommendations

**Remediation:**

Emit dedicated events for all privileged configuration changes, where useful, include both old and new values in emitted events.

**Resolution:****Fixed in 5377ac5 :**

Dedicated events have been added for the privileged **UniversalCore** setters identified in the finding, and `setName` and `setSymbol` now emit `NameUpdated` and `SymbolUpdated` events with old and new values. The missing event-emission gap for the affected configuration and metadata updates is no longer present.

## [F-2026-15528](#) - PC Can Become Permanently Stuck Due to Missing Rescue Path - Info

**Description:** **UniversalCore** implements a `payable receive` function and can therefore accept native PC. However, the contract does not provide any admin recovery or rescue mechanism for accidentally sent native funds that are not part of the intended execution flow.

```
/// @notice Accept native PC transfers (e.g., from WPC withdraw).
receive() external payable {}
```

Native PC sent directly to the contract outside normal protocol flows may become permanently unrecoverable.

This creates operational loss risk for users, integrators, or internal operators who transfer funds to the contract by mistake.

**Assets:**

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Fixed

### Classification

**Impact Rate:** 3/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Consider adding an admin-governed rescue function for accidental native-token balances, for example a restricted withdrawal path with clear access controls and event emission.

**Resolution:** Fixed in `b41f174` :

**UniversalCore** now includes an admin-only `rescueNativePC(address payable to, uint256 amount)` function with zero-address, zero-amount, balance, and

transfer-success checks, plus a **RescueNativePC** event. Accidental native-PC transfers are no longer permanently unrecoverable.

## F-2026-15529 - CEA Multicall Does Not Enforce Aggregate Value Bounds - Info

### Description:

`_handleMulticall` executes each call with its individually specified value, but does not validate the aggregate value across the batch. As a result, the total ETH/native value consumed by the multicall can exceed the `msg.value` supplied to `executeUniversalTx`, causing the batch to draw on any pre-existing native balance held by the CEA.

```
function _handleMulticall(bytes32 txId, bytes32 universalTxId, address origin
Caller, Multicall[] memory calls)
    internal
{
    for (uint256 i = 0; i < calls.length; i++) {
        if (calls[i].to == address(0)) {
            revert CEAEerrors.InvalidTarget();
        }

        if (calls[i].to == address(this) && calls[i].value != 0) {
            revert CEAEerrors.InvalidInput();
        }

        (bool success,) = calls[i].to.call{value: calls[i].value}(calls[i].da
ta);

        if (!success) revert CEAEerrors.ExecutionFailed();

        emit UniversalTxExecuted(txId, universalTxId, originCaller, calls[i].
to, calls[i].data);
    }
}
```

A multicall can spend not only the ETH/native value attached to the current execution, but also any native balance already present on the CEA.

### Assets:

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Accepted

### Classification

### Impact Rate:

1/5



|                         |             |
|-------------------------|-------------|
| <b>Likelihood Rate:</b> | 1/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

## Recommendations

**Remediation:** Track cumulative `calls[i].value` and enforce an upper bound, for example against:

- `msg.value`, if the intention is to limit spending to the current execution, or
- an explicitly documented balance policy if spending present funds is intended.

## Resolution:

### Accepted:

The behavior is intentional and consistent with CEA's architectural role as a persistent custody account. The Vault deposits native funds via the park-funds path (`payload.length == 0 && recipient == address(0)`), and subsequent multicalls are designed to spend from the accumulated balance. Enforcing aggregate-value bounds against `msg.value` would strand parked funds and create asymmetry with the single-call and ERC-20 paths, which exhibit identical balance-spending semantics. The `onlyVault` modifier combined with `originCaller == pushAccount` validation ensures no external actor can trigger unauthorized multicalls, making the access-control layer the effective security boundary rather than per-call value accounting.

## F-2026-15537 - uniswapV3Quoter Is Stored but Never Used - Info

### Description:

**UniversalCore** stores a configurable `uniswapV3Quoter` address and allows admins to update it, but the value is never referenced anywhere in contract logic. As a result, the protocol exposes configuration that appears operationally relevant while having no effect on runtime behavior.

```
address public uniswapV3Factory;  
address public uniswapV3SwapRouter;  
address public uniswapV3Quoter;  
  
...  
  
WPC = wpc_;  
uniswapV3Factory = uniswapV3Factory_;  
uniswapV3SwapRouter = uniswapV3SwapRouter_;  
uniswapV3Quoter = uniswapV3Quoter_;  
  
...  
  
function setUniswapV3Addresses(address factory, address swapRouter, address q  
uoter) external onlyAdmin {  
    if (factory == address(0) || swapRouter == address(0) || quoter == address  
(0)) {  
        revert CommonErrors.ZeroAddress();  
    }  
    uniswapV3Factory = factory;  
    uniswapV3SwapRouter = swapRouter;  
    uniswapV3Quoter = quoter;  
}
```

This creates a configuration/behavior inconsistency: operators may assume quoter configuration affects swap pricing or slippage logic, but it does not. This increases maintenance and review overhead by introducing dead protocol state.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 1/5         |
| <b>Likelihood Rate:</b> | 2/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

---

## Recommendations

- Remediation:** Either:
- integrate `uniswapV3Quoter` into quote/slippage logic if it is intended to be part of swap validation, or
  - remove the state variable and related configuration path if it is not required.

**Resolution:** **Fixed in** `1fe77ca` :

The unused `uniswapV3Quoter` state variable and its related configuration paths were removed from **UniversalCore**, including the initializer parameter and the `setUniswapV3Addresses` setter argument. The current implementation no longer exposes non-functional quoter configuration.

## [F-2026-15540](#) - Inconsistent Zero-Value Handling for Base and Rescue Gas Limits - Info

### Description:

**UniversalCore** applies inconsistent validation rules to two analogous chain gas-limit configurations:

- `rescueFundsGasLimitByChainNamespace[chainNamespace] == 0` causes an explicit revert in `getRescueFundsGasLimit`
- `baseGasLimitByChainNamespace[chainNamespace] == 0` is silently accepted in `getOutboundTxGasAndFees`, which can then produce a zero gas fee when fallback logic is used

`getOutboundTxGasAndFees`:

```
chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();
uint256 baseLimit = baseGasLimitByChainNamespace[chainNamespace];
if (gasLimitWithBaseLimit == 0) {
    gasLimitWithBaseLimit = baseLimit;
} else if (gasLimitWithBaseLimit < baseLimit) {
    revert UniversalCoreErrors.GasLimitBelowBase(gasLimitWithBaseLimit, baseLimit);
}
```

`getRescueFundsGasLimit`:

```
chainNamespace = IPRC20(_prc20).SOURCE_CHAIN_NAMESPACE();
rescueGasLimit = rescueFundsGasLimitByChainNamespace[chainNamespace];
if (rescueGasLimit == 0) {
    revert UniversalCoreErrors.ZeroRescueGasLimit();
}
```

Two similarly purposed configuration values are treated differently at runtime, which makes system behavior less predictable. Integrators and operators may expect unset gas-limit configuration to fail consistently across both quote paths, but only the rescue path enforces this.

This inconsistency increases configuration ambiguity and can lead to incorrect assumptions during chain setup or troubleshooting.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

## Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 1/5         |
| <b>Likelihood Rate:</b> | 2/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

---

## Recommendations

**Remediation:** Align the validation model for both gas-limit types. For example:

- either revert when `baseGasLimitByChainNamespace[chainNamespace] == 0`, or
- explicitly document that `baseGasLimit == 0` is a valid configuration with distinct semantics.

**Resolution:** **Fixed in 9794d51 :**

`getOutboundTxGasAndFees` now explicitly reverts with `ZeroBaseGasLimit` when `baseGasLimitByChainNamespace[chainNamespace]` is unset, aligning its behavior with `getRescueFundsGasLimit` for zero-valued gas-limit configuration. The runtime inconsistency described in the finding is no longer present.

## [F-2026-15542](#) - chainHeightByChainNamespace Is Stored but Unused - Info

**Description:** **UniversalCore** stores per-chain observed block height in `chainHeightByChainNamespace`, but the value is never consumed by any runtime logic. It is written in `setChainMeta` and exposed via the public getter only.

```
mapping(string => uint256) public chainHeightByChainNamespace;
mapping(string => uint256) public timestampObservedAtByChainNamespace;

function setChainMeta(string memory chainNamespace, uint256 price, uint256 chainHeight)
    external
    onlyUEModule
{
    gasPriceByChainNamespace[chainNamespace] = price;
    chainHeightByChainNamespace[chainNamespace] = chainHeight;
    timestampObservedAtByChainNamespace[chainNamespace] = block.timestamp;
    emit SetChainMeta(chainNamespace, price, chainHeight, block.timestamp);
}
```

This adds protocol state that appears meaningful but has no effect on execution or validation and can mislead reviewers and operators into assuming chain-height-based freshness or ordering checks exist when they do not.

**Assets:**

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Accepted

### Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 2/5         |
| <b>Likelihood Rate:</b> | 1/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

### Recommendations

**Remediation:**

Either:

- incorporate `chainHeightByChainNamespace` into actual validation or monitoring logic, or
- remove it if it is only leftover telemetry and not required by downstream systems.

If it is intended purely for off-chain observability, document that clearly.

**Resolution:****Accepted:**

`chainHeightByChainNamespace` is intentionally retained for off-chain observability rather than on-chain enforcement. Explicit NatSpec clarifying that the value is informational-only and not consumed by runtime validation adequately addresses the ambiguity identified in the finding, while the absence of on-chain use remains a deliberate design choice.

## [F-2026-15543](#) - Role Management Surface for PAUSER\_ROLE Is Inconsistent and Potentially Misleading - Info

### Description:

The **UniversalCore**, **CEAFactory** and **UEAFactory** contracts expose a dedicated helper `setPauserRole` function, which suggests a canonical and possibly exclusive way to configure the pauser role.

```
function setPauserRole(address newPauser) external onlyAdmin {
    if (newPauser == address(0)) revert CommonErrors.ZeroAddress();
    _grantRole(PAUSER_ROLE, newPauser);
    emit PauserRoleGranted(newPauser);
}
```

However, because contracts inherit **AccessControlUpgradeable**, admins can also grant or revoke `PAUSER_ROLE` directly through the generic `grantRole` / `revokeRole` functions.

The interface creates the impression that `setPauserRole` is the intended or exclusive configuration path, while in reality role assignment can also occur through inherited generic access-control functions.

The `set...` naming convention may suggest replacement semantics, but the function actually performs additive role assignment, which can mislead operators into believing the previous pauser is rotated out when it is not.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- CEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Info



## Recommendations

### Remediation:

Consider one of the following:

- rename the helper to reflect additive semantics, e.g. `grantPauserRole`,
- provide explicit rotation helpers such as `replacePauser(oldPauser, newPauser)`,
- emit/document a clear runbook that `grantRole` / `revokeRole` remain valid admin paths,
- standardize role-management patterns across the codebase to avoid mixed semantics and monitoring gaps.

### Resolution:

**Fixed in** `41ebc6a` :

The dedicated `setPauserRole` helper has been removed from `UniversalCore`, `CEAFactory`, and `UEAFactory`, along with the related interface declarations and custom `PauserRoleGranted` events. Role administration for `PAUSER_ROLE` now relies on the inherited OpenZeppelin `grantRole` and `revokeRole` functions, eliminating the misleading additive `set...` helper semantics identified in the finding.

## F-2026-15544 - Stored Gas Pool Configuration Diverges from Runtime Swap Resolution - Info

### Description:

`setGasPCPool` stores a pool address for a given chain namespace after validating its existence. However, runtime swap flows do not use the stored pool at all. Instead, they resolve the pool dynamically from the factory each time using the current token pair and fee tier.

```
function setGasPCPool(string memory chainNamespace, address gasToken, uint24
fee) external onlyRole(MANAGER_ROLE) {
    if (gasToken == address(0)) revert CommonErrors.ZeroAddress();
    address pool = IUniswapV3Factory(uniswapV3Factory)
        .getPool(WPC < gasToken ? WPC : gasToken, WPC < gasToken ? gasToken :
WPC, fee);
    if (pool == address(0)) revert UniversalCoreErrors.PoolNotFound();
    gasPCPoolByChainNamespace[chainNamespace] = pool;
    emit SetGasPCPool(chainNamespace, pool, fee);
}
```

But runtime logic does not reference `gasPCPoolByChainNamespace`:

`swapAndBurnGas`:

```
address pool = IUniswapV3Factory(uniswapV3Factory)
    .getPool(WPC < gasToken ? WPC : gasToken, WPC < gasToken ? gasToken : WPC,
fee);
if (pool == address(0)) revert UniversalCoreErrors.PoolNotFound();
```

`_autoSwap`:

```
address pool = IUniswapV3Factory(uniswapV3Factory)
    .getPool(prc20 < WPC ? prc20 : WPC, prc20 < WPC ? WPC : prc20, resolvedFee
);
if (pool == address(0)) revert UniversalCoreErrors.PoolNotFound();
```

The contract exposes a stored gas-pool configuration that appears operationally meaningful, but runtime behavior ignores it.

This creates a configuration/runtime mismatch and can mislead operators into thinking pool selection is pinned by stored state when it is actually resolved dynamically.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Mitigated

---

## Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Choose one model and apply it consistently:

- either use `gasPCPoolByChainNamespace` in swap logic, or
- remove the stored pool configuration if pool selection is intended to be fully dynamic.

If it is retained only for observability, document that it is informational and not enforced at runtime.

**Resolution:** **Mitigated in** `68cb748` :

The stored `gasPCPoolByChainNamespace` value remains unused by runtime swap execution, so the underlying configuration/runtime mismatch still exists. Commit `68cb748` reduced the risk by adding explicit NatSpec stating that `gasPCPoolByChainNamespace` and `setGasPCPool` are informational-only for off-chain observability and are not enforced at runtime.

## [F-2026-15545](#) - Use of Magic Numbers Reduces Clarity and Increases Maintenance Risk in UniversalCore - Info

### Description:

**UniversalCore** contains several hardcoded numeric constants and sentinel values embedded directly in logic instead of using named constants or enums. While this does not currently create a direct exploit path, it reduces readability, makes intent less explicit, and increases the risk of inconsistent updates in future changes.

`setDefaultFeeTier`:

```
if (feeTier != 500 && feeTier != 3000 && feeTier != 10000) {  
    revert UniversalCoreErrors.InvalidFeeTier();  
}
```

`setSlippageTolerance`:

```
if (tolerance > 5000) {  
    revert UniversalCoreErrors.InvalidSlippageTolerance();  
}
```

It obscures business meaning of values such as supported Uniswap fee tiers, slippage caps, and operational deadlines.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

### Recommendations

### Remediation:

Replace repeated or protocol-significant literals with named constants.

**Resolution:****Fixed in 295749c :**

Named constants were introduced for supported Uniswap fee tiers and the slippage cap, and the hardcoded literals in `setDefaultFeeTier` and `setSlippageTolerance` were replaced accordingly. The readability and maintainability issue described in the finding is no longer present.

## [F-2026-15549](#) - UniversalCore Documentation Is Out of Sync With Implementation - Info

### Description:

The **UniversalCore** documentation contains several interface and behavior descriptions that no longer match the actual contract implementation. This creates a specification/implementation mismatch that can mislead integrators, operators, and reviewers about how privileged flows and fee settlement actually work.

1. `setChainMeta` signature: the documentation describes a four-parameter function, including `observedAt`, while the implementation accepts only three parameters and derives observation time internally from `block.timestamp`.
2. `swapAndBurnGas` signature and flow: the documentation describes `swapAndBurnGas` as taking `vault` and `protocolFee` parameters and emitting them in the event flow, but the implementation does neither.
3. Pause authority: the documentation places `pause` / `unpause` under the admin role, while the implementation restricts both to `PAUSER_ROLE`.
4. Function naming and parameter simplification: the documentation references functions and signatures that do not exist as written, such as:
  - `setWPCContractAddress(addr)` instead of implemented `setWPC(addr)`
  - simplified `refundUnusedGas(recipient, amount)` instead of the actual six-parameter function

Integrators may implement against incorrect function signatures or incorrect fee-settlement assumptions.

Operators and reviewers may misunderstand trust boundaries, especially around pausing and protocol-fee handling.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Update **docs/UniversalCore.md** to reflect the deployed implementation precisely, especially for:

- function signatures,
- event semantics,
- fee-settlement flow,
- role ownership of privileged actions.

Or update code parts which should follow the mentioned documentation.

**Resolution:** **Fixed in** 0f8e49d :

`docs/UniversalCore.md` has been synchronized with the current implementation, including the three-parameter `setChainMeta`, the five-parameter `swapAndBurnGas` flow, `PAUSER_ROLE` ownership of `pause` / `unpause`, and the actual `setWPC` and six-parameter `refundUnusedGas` signatures. The specification/implementation mismatches described in the finding are no longer present.

## [F-2026-15550](#) - Leading-Zero Chain IDs Create Ambiguous Numeric Interpretation - Info

### Description:

`stringToExactUInt256` function from **StringUtils** accepts decimal strings with leading zeros (e.g. "1", "01", "001") and converts all of them to the same numeric value. In the UEA system, this creates an inconsistency because factory deployment identity uses the raw string value, while the EIP-712 domain separator converts that same string into a normalized integer.

```
function stringToExactUInt256(
    string memory s
) internal pure returns (uint256 result) {
    bytes memory b = bytes(s);
    uint256 len = b.length;
    require(len > 0, "Empty string cannot be converted.");
    for (uint256 i = 0; i < len; ++i) {
        uint8 c = uint8(b[i]);
        require(c >= 48 && c <= 57, "Non-digit character found.");
        result = result * 10 + (c - 48);
    }
}
```

The deployment identity keeps the original string:

```
function generateSalt(UniversalAccountId memory _id) public pure returns (bytes32) {
    return keccak256(abi.encode(_id));
}
```

But the EVM UEA domain separator normalizes it to `uint256`:

```
function domainSeparator() public view returns (bytes32) {
    uint256 chainId = StringUtils.stringToExactUInt256(
        _universalAccountId.chainId
    );
    return keccak256(
        abi.encode(
            DOMAIN_SEPARATOR_TYPEHASH,
            keccak256(bytes(VERSION)),
            chainId,
            address(this)
        )
    );
}
```



Distinct textual identities such as "1" and "01" produce different deployment salts but the same numeric chain ID in the EIP-712 domain.

This creates inconsistent identity handling across deployment and signature logic.

The mismatch increases the risk of integration bugs, identity confusion, and incorrect assumptions about how chain IDs are canonicalized.

#### Assets:

- Utils.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Accepted

#### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 1/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

#### Recommendations

**Remediation:** Canonicalize chain IDs consistently across the system. For example:

- reject leading-zero decimal strings unless the value is exactly "0", or
- normalize chain IDs before both deployment-salt generation and domain-separator construction.

Also document the expected canonical chain ID format clearly for all integrators.

#### Resolution:

##### Accepted:

The current implementation still permits leading-zero decimal strings in `stringToExactUInt256`, while `generateSalt` continues to derive deployment identity from the raw `chainId` string and `UEA_EVM.domainSeparator` normalizes that same string to `uint256`. The repository's tests explicitly preserve this behavior by asserting that values such as "001" convert to 1, and no canonicalization or leading-zero rejection has been added, indicating that the mismatch remains an acknowledged design choice rather than a remediated issue.

## [F-2026-15556](#) - Return Values from IPRC20.deposit and IPRC20.approve Are Ignored - Info

**Description:** **UniversalCore** calls **IPRC20** functions that return bool, but does not validate the returned value.

This occurs in multiple places:

- `depositPRC20Token`
  - `IPRC20(prc20).deposit(recipient, amount);`
- `refundUnusedGas (withSwap == false)`
  - `IPRC20(gasToken).deposit(recipient, amount);`
- `_autoSwap`
  - `IPRC20(prc20).deposit(address(this), amount);`
  - `IPRC20(prc20).approve(uniswapV3SwapRouter, amount);`
  - `IPRC20(prc20).approve(uniswapV3SwapRouter, 0);`

The functions rely on the assumption that the token implementation either reverts on failure or always returns true.

If a future or alternate **PRC20**-compatible token returns false instead of reverting, **UniversalCore** may continue execution under a false-success assumption (especially dangerous around `_autoSwap` preparation), causing inconsistent state transitions or unexpected downstream behavior.

This weakens interface robustness and makes the calling code more tightly coupled to the current **PRC20** implementation details.

### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

## Recommendations

### Remediation:

Either:

- check returned boolean values explicitly, or
- document that `IPRC20` implementations used by **UniversalCore** must revert on failure and never return `false`.

### Resolution:

**Fixed in** `5ff49ed` :

**UniversalCore** now checks the return values of `IPRC20.deposit` and `IPRC20.burn` and reverts with `PRC200operationFailed` if a token returns `false` instead of reverting. The `_autoSwap` approval path cited in the finding is no longer present in its original form because router approvals were migrated to `SafeERC20.forceApprove`, so the unchecked-bool pattern described in the finding no longer exists in the current implementation.

## F-2026-15562 - SafeERC20 Is Imported but Never Used in CEA - Info

### Description:

**CEA** imports **SafeERC20** and attaches it to **IERC20**, but no `safe*` methods are actually used anywhere in the contract. The only ERC-20 interaction performed is `balanceOf`, which does not use the library.

```
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

contract CEA is ICEA, ReentrancyGuard {
    using SafeERC20 for IERC20;
}
```

This creates dead import / dead abstraction noise and may mislead readers into expecting defensive ERC-20 handling that is not actually present.

### Assets:

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

## Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 1/5         |
| Likelihood Rate: | 1/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

## Recommendations

### Remediation:

- remove the unused import and `using SafeERC20 for IERC20`, or
- adopt **SafeERC20** consistently if future token transfers/approvals are expected in this contract.

## Resolution:

## Fixed in 706f3c1 :

The unused `SafeERC20` import and `using SafeERC20 for IERC20` directive have been removed from **CEA**. The dead abstraction and misleading defensive-safety signal described in the finding are no longer present.

## F-2026-15563 - Proxy Initialization Validation Is Inconsistent Between CEAProxy and UEAProxy - Info

### Description:

The two proxy contracts follow nearly identical initialization patterns, but they apply different validation rules to the implementation address.

`initializeCEAProxy` function from **CEAProxy** explicitly rejects `address(0)`, while `initializeUEA` from **UEAProxy** does not validate `_logic` before storing it.

### CEAProxy:

```
function initializeCEAProxy(address _logic) external initializer {
    if (_logic == address(0)) {
        revert CEAEErrors.InvalidCall();
    }

    address currentImpl = getImplementation();
    if (currentImpl != address(0)) {
        revert CEAEErrors.InvalidCall();
    }
}
```

### UEAProxy:

```
function initializeUEA(address _logic) external initializer {
    address currentImpl = getImplementation();
    if (currentImpl != address(0)) {
        revert UEAErrors.InvalidCall();
    }

    assembly {
        sstore(UEA_LOGIC_SLOT, _logic)
    }
}
```

Similar proxy components enforce different safety assumptions, which increases maintenance and review complexity.

The inconsistency can mislead developers into assuming both initialization paths have equivalent validation guarantees.

While factory flows may currently prevent bad inputs, the proxy-level validation surface is not uniform.

### Assets:

- CEAProxy.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- UEAProxy.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Fixed

---

## Classification

**Impact Rate:** 2/5

**Likelihood Rate:** 1/5

**Exploitability:** Semi-Dependent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Standardize validation behavior across both proxy contracts.

If zero-address initialization should always be invalid, add the same explicit check to **UEAProxy**.

If factory-only guarantees are considered sufficient, document that assumption consistently.

**Resolution:** Fixed in 820068a :

`initializeUEA` now rejects `address(0)` for `_logic` before storing the implementation address, aligning its initialization validation with **CEAProxy**. The proxy-level inconsistency described in the finding is no longer present.

## [F-2026-15564](#) - CEA Multicall Discards Underlying Revert Reasons - Info

**Description:** In `_handleMulticall`, each subcall is executed with low-level call, but if a subcall fails the contract reverts with the generic custom error `ExecutionFailed` from **CEAErrors** instead of propagating the original revert data from the failed target.

```
(bool success,) = calls[i].to.call{value: calls[i].value}(calls[i].data);  
if (!success) revert CEAErrors.ExecutionFailed();  
emit UniversalTxExecuted(txId, universalTxId, originCaller, calls[i].to, calls[i].data);
```

Debugging failed multicalls becomes significantly harder because the original failure cause is lost.

Off-chain monitoring, integration testing, and operational troubleshooting receive less actionable information.

This is especially relevant in a contract designed to execute arbitrary external calls, where revert specificity is valuable.

**Assets:**

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 2/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** Capture and arise the returned revert data when available, similar to the pattern used in the **UEA** execution flow.



**Resolution:****Fixed in `b7d1f23` :**

`_handleMulticall` now captures revert data from failed low-level calls and reverts with the original payload when available, falling back to `ExecutionFailed` only when revert data is empty. The loss of underlying failure context described in the finding is no longer present.

## F-2026-15576 - getOriginForUEA Returns Synthetic Identity With Hardcoded Push Chain ID - Info

### Description:

The `getOriginForUEA` function from **UEAFactory** returns the registered `UniversalAccountId` for a deployed **UEA**. However, when the input address is not recognized as a **UEA**, the function fabricates a fallback identity using a hardcoded chain identifier "`42101`":

```
function getOriginForUEA(address addr) external view returns (UniversalAccount  
tId memory account, bool isUEA) {  
    account = UEA_to_UOA[addr];  
    if (account.owner.length > 0) {  
        isUEA = true;  
    } else {  
        account =  
            UniversalAccountId({chainNamespace: "eip155", chainId: "42101", ow  
ner: bytes(abi.encodePacked(addr))});  
    }  
    return (account, isUEA);  
}
```

This fallback path assumes that any non-UEA address should be represented as a Push-native identity on chain ID "`42101`". That value is embedded directly in code and cannot be updated through configuration.

This creates two related issues:

- the returned identity may become incorrect if Push Chain uses a different chain ID across environments or after future network changes;
- downstream integrations may incorrectly treat the fabricated identity as authoritative if they read account but ignore the accompanying `isUEA` flag.

### Assets:

- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

---

## Recommendations

**Remediation:** Avoid hardcoding the Push Chain ID in fallback identity construction.  
Possible remediations:

- return an empty/default UniversalAccountId when `isUEA == false`, forcing callers to handle the non-UEA case explicitly; or
- make the Push Chain identity parameters configurable and clearly document that this branch returns a synthetic fallback identity rather than a registered UEA origin.

**Resolution:** Fixed in 8517898 :

The hardcoded fallback chain ID in `getOriginForUEA` has been replaced with the configurable `pushChainId` state variable, seeded during initialization and updateable by admin. Additional NatSpec now clarifies that the fallback branch returns a synthetic identity and that `isUEA` must be checked before treating the returned account as authoritative.

## F-2026-15577 - registerUEA Can Overwrite Existing VM

### Implementation Mapping - Info

#### Description:

The `registerUEA` function from **UEAFactory** updates the implementation address associated with a VM type hash. Although it validates that the provided `_chainHash` is registered and mapped to the supplied `_vmHash`, it does not prevent overwriting an already configured implementation for that VM type:

```
function registerUEA(bytes32 _chainHash, bytes32 _vmHash, address _UEA) public
onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_UEA == address(0)) {
        revert UEAErrors.InvalidInputArgs();
    }
    (bytes32 registeredVmHash, bool isRegistered) = getVMType(_chainHash);
    if (!isRegistered || registeredVmHash != _vmHash) {
        revert UEAErrors.InvalidInputArgs();
    }
    UEA_VM[_vmHash] = _UEA;
    emit UEARegistered(_chainHash, _UEA, _vmHash);
}
```

As a result, any later call to `registerUEA` for the same `_vmHash` silently replaces the prior implementation address.

This creates a mutable shared configuration surface for all future UEA deployments of that VM type:

- an admin can accidentally replace the implementation for an existing VM family;
- future UEA deployments may begin using a different implementation than earlier deployments without any explicit “update” workflow;
- the name `registerUEA` suggests one-time registration, but the actual behavior is replacement;
- off-chain tooling or reviewers may incorrectly assume VM implementation mappings are append-only.

#### Assets:

- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Fixed

#### Classification

|                         |           |
|-------------------------|-----------|
| <b>Impact Rate:</b>     | 2/5       |
| <b>Likelihood Rate:</b> | 2/5       |
| <b>Exploitability:</b>  | Dependent |
| <b>Complexity:</b>      | Simple    |
| <b>Severity:</b>        | Info      |

## Recommendations

**Remediation:** If overwriting is intended, expose it through a clearly named update function such as `setUEAImplementation` and emit an event that includes both old and new values.

If overwriting is not intended, reject reassignment when `UEA_VM[_vmHash]` is already set:

```
if (UEA_VM[_vmHash] != address(0)) {
    revert UEAEErrors.InvalidInputArgs();
}
```

**Resolution:** **Fixed in 9c3b78a :**

`registerUEA` now reverts with `UEAAlreadyRegistered` when an implementation already exists for the given VM hash, preventing silent overwrite of an existing mapping. An explicit `updateUEAImplementation` admin function and `UEAImplementationUpdated` event were added to handle intentional replacements transparently, eliminating the ambiguous replacement behavior described in the finding.

## F-2026-15593 - getOriginForUEA returns synthetic identity that may confuse authorization logic - Info

### Description:

When `addr` is not a registered/deployed UEA, `getOriginForUEA` does not return an empty identity. Instead, it fabricates a fallback `UniversalAccountId`:

- `chainNamespace` = `"eip155"`
- `chainId` = `"42101"` (hardcoded)
- `owner` = `bytes(abi.encodePacked(addr))`

while separately setting `isUEA = false`.

```
function getOriginForUEA(address addr) external view returns (UniversalAccount  
tId memory account, bool isUEA) {  
    account = UEA_to_UOA[addr];  
  
    if (account.owner.length > 0) {  
        isUEA = true;  
    } else {  
        account = UniversalAccountId({chainNamespace: "eip155", chainId: "421  
01", owner: bytes(abi.encodePacked(addr))});  
    }  
  
    return (account, isUEA);  
}
```

So for unknown addresses, the function still returns a valid-looking origin object that can be mistaken for canonical identity data.

If integrators/other contracts consume only `account` and forget to enforce `isUEA`, they may:

- treat arbitrary addresses as valid UEA-origin identities,
- apply incorrect authorization/routing rules,
- silently accept synthetic identity in place of registered mapping data.

This is primarily an integration/auth confusion risk (not direct on-chain takeover).

### Assets:

- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Mitigated

## Classification

|                         |             |
|-------------------------|-------------|
| <b>Impact Rate:</b>     | 2/5         |
| <b>Likelihood Rate:</b> | 1/5         |
| <b>Exploitability:</b>  | Independent |
| <b>Complexity:</b>      | Simple      |
| <b>Severity:</b>        | Info        |

---

## Recommendations

### Remediation:

- Prefer explicit failure semantics for unknown addresses (revert or return empty struct), or
- keep current behavior but make it unambiguous:
  - rename to indicate fallback behavior (e.g., `getOriginOrSynthetic`),
  - document strongly that callers must check `isUEA`,

### Resolution:

#### Mitigated in 8517898 :

The synthetic fallback behavior in `getOriginForUEA` remains present, so unknown addresses still return a valid-looking `UniversalAccountId` alongside `isUEA = false`. Commit 8517898 reduced the integration risk by replacing the hardcoded chain ID with configurable `pushChainId` and adding explicit NatSpec stating that the fallback result is synthetic and that callers must check `isUEA` before trusting the returned account.

## [F-2026-15616](#) - initializeCEA Arguments Differ From Documented Specification - Info

**Description:** The `initializeCEA` function in the `CEA` contract is implemented with four function arguments.

```
function initializeCEA(address _pushAccount, address _vault, address _universalGateway, address _factory) external {
```

However, the documentation in `3_CEA.md` defines the `initializeCEA` function with three arguments.

```
3. `CEAFactorY` calls `CEA.initializeCEA(pushAccount, VAULT, UNIVERSAL_GATEWAY)` through the proxy.
```

This results in an inconsistency between the documented specification and the on-chain implementation. Such a mismatch may lead to incorrect assumptions when interacting with the contract or interpreting its initialization requirements.

**Assets:**

- CEA.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:** Fixed

### Classification

**Impact Rate:** 1/5

**Likelihood Rate:** 5/5

**Exploitability:** Independent

**Complexity:** Simple

**Severity:** Info

### Recommendations

**Remediation:** It is recommended to align the documentation with the implemented function signature or update the implementation to match the documented specification. Consistency between documentation and code ensures



accurate interpretation of the contract interface and reduces the risk of integration errors.

## Resolution:

### Fixed in `fd12011`:

The CEA documentation has been updated to match the implemented four-argument `initializeCEA(pushAccount, VAULT, UNIVERSAL_GATEWAY, factory)` initialization flow. The specification/implementation mismatch described in the finding is no longer present.

## F-2026-15617 - Gas Inefficiency Due To Dynamic String Mapping

### Keys - Info

#### Description:

The **UniversalCore** contract uses multiple `mapping(string => ...)` mappings for chain-related settings. The use of dynamic `string` keys introduces additional gas overhead because each access requires dynamic memory allocation and hashing of variable-length data.

```
mapping(string => uint256) public gasPriceByChainNamespace;  
mapping(string => address) public gasTokenPRC20ByChainNamespace;  
mapping(string => uint256) public baseGasLimitByChainNamespace;  
mapping(string => uint256) public rescueFundsGasLimitByChainNamespace;  
mapping(string => uint256) public chainHeightByChainNamespace;  
mapping(string => uint256) public timestampObservedAtByChainNamespace;
```

Compared to fixed-size types such as `bytes32`, dynamic strings increase computational cost during both read and write operations. This results in higher gas consumption for all interactions involving these mappings. Additionally, the use of dynamic keys contributes to increased complexity in data handling and may lead to inefficient state usage over time.

#### Assets:

- UniversalCore.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

#### Status:

Accepted

### Classification

|                  |             |
|------------------|-------------|
| Impact Rate:     | 1/5         |
| Likelihood Rate: | 5/5         |
| Exploitability:  | Independent |
| Complexity:      | Simple      |
| Severity:        | Info        |

### Recommendations

#### Remediation:

It is recommended to replace `string` mapping keys with fixed-size types such as `bytes32`. This can be achieved by hashing or encoding identifiers into a fixed-length format prior to storage.

## Resolution:

## Accepted:

**UniversalCore** continues to use `mapping(string => ...)` for chain-scoped configuration, and no remediation is present that replaces CAIP-2 string namespaces with fixed-size identifiers such as `bytes32`. The retained design matches the documented protocol model, which treats chain identifiers as namespace strings, indicating an intentional tradeoff in favor of direct CAIP-2 compatibility despite the additional gas overhead.

## [F-2026-15618](#) - Redundant Role Check in Batch Registration

### Increases Gas Costs - Info

#### Description:

`registerMultipleUEA` function from the `UEAFactory` performs batch registration by iterating over an array and invoking the public `registerUEA` function on each iteration. Since `registerUEA` is protected by `onlyRole(DEFAULT_ADMIN_ROLE)`, The role verification is executed repeatedly for every element in the loop, resulting in avoidable overhead during batch operations.

```
function registerMultipleUEA(bytes32[] memory _chainHashes, bytes32[] memory
_vmHashes,
                                address[] memory _UEA) external onlyRole(DEFAULT
_ADMIN_ROLE) {
    if (_UEA.length != _vmHashes.length || _UEA.length != _chainHashes.length
) {
        revert UEAErrors.InvalidInputArgs();
    }

    for (uint256 i = 0; i < _UEA.length; i++) {
        registerUEA(_chainHashes[i], _vmHashes[i], _UEA[i]);
    }
}

function registerUEA(bytes32 _chainHash, bytes32 _vmHash,
                                address _UEA) public onlyRole(DEFAULT_ADMIN_ROLE) {
    if (_UEA == address(0)) {
        revert UEAErrors.InvalidInputArgs();
    }

    (bytes32 registeredVmHash, bool isRegistered) = getVMType(_chainHash);
    if (!isRegistered || registeredVmHash != _vmHash) {
        revert UEAErrors.InvalidInputArgs();
    }

    UEA_VM[_vmHash] = _UEA;
    emit UEARegistered(_chainHash, _UEA, _vmHash);
}
```

This pattern introduces a gas inefficiency, as authorization is already enforced at the batch entry point. Calling a role-restricted public function within the loop leads to duplicated access control checks and increased per-item execution cost.

**Assets:**

- UEAFactory.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

**Status:****Fixed****Classification****Impact Rate:**

1/5

**Likelihood Rate:**

5/5

**Exploitability:**

Independent

**Complexity:**

Simple

**Severity:****Info****Recommendations****Remediation:**

It is recommended to enforce authorization only at the batch entry point and process each element through an internal function that performs per-item validation and state updates without repeating role checks. This approach maintains the same security guarantees while reducing gas consumption during batch execution.

**Resolution:****Fixed in** `0a257ac` :

The registration logic was extracted into internal `_registerUEA`, and `registerMultipleUEA` now calls that internal function directly rather than the role-protected public `registerUEA`. Repeated `onlyRole(DEFAULT_ADMIN_ROLE)` checks inside the batch loop have been eliminated.

## F-2026-15619 - Custom Errors Can Be Used for Gas Efficiency - Info

### Description:

Starting from Solidity version `0.8.4`, the language introduced a feature known as "custom errors". These custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Prior to this version, developers often used the `require` statement with string error messages to handle specific conditions or validations. However, every unique string used as a revert reason consumes gas, making transactions more expensive.

Custom errors, on the other hand, are identified by their name and the types of their parameters only, and they do not have the overhead of string storage. This means that, when using custom errors instead of `require` statements with string messages, the gas consumption can be significantly reduced, leading to more gas-efficient contracts.

### Assets:

- WPC.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]
- Utils.sol [<https://github.com/pushchain/push-chain-core-contracts/tree/audit-main>]

### Status:

Fixed

### Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

### Recommendations

#### Remediation:

It is recommended to use custom errors instead of revert strings to reduce increased Gas usage, especially during contract deployment. Custom errors can be defined using the `error` keyword and can include dynamic information.

#### Resolution:

Fixed in `58b3554` :

Within the audit scope, revert strings have been replaced with custom errors, removing the gas-inefficient string-based error pattern described in the finding. Out-of-scope mock contracts do not affect this classification.

## Disclaimers

### Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

### Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.



## Appendix 1. Definitions

### Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

| Severity | Description                                                                                                                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Critical | Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.                                                                                       |
| High     | High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.                                |
| Medium   | Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category. |
| Low      | Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.                                                                                           |

### Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

## Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

| Scope Details          |                                                                                                                                                                                                                                   |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository             | <a href="https://github.com/pushchain/push-chain-core-contracts/tree/audit-main">https://github.com/pushchain/push-chain-core-contracts/tree/audit-main</a>                                                                       |
| Commit                 | 15b70544feba1586f488a90e38650bf33ea3c634                                                                                                                                                                                          |
| Remediation commit     | daf74825d22121896964e2d542201e623c4e994c                                                                                                                                                                                          |
| Post Audit Changes     | a2b61850d765679aa9e1e39998a47a348404bc08                                                                                                                                                                                          |
| Whitepaper             | -                                                                                                                                                                                                                                 |
| Requirements           | <a href="https://github.com/pushchain/push-chain-core-contracts/tree/15b70544feba1586f488a90e38650bf33ea3c634/docs">https://github.com/pushchain/push-chain-core-contracts/tree/15b70544feba1586f488a90e38650bf33ea3c634/docs</a> |
| Technical Requirements | <a href="https://github.com/pushchain/push-chain-core-contracts/tree/15b70544feba1586f488a90e38650bf33ea3c634/docs">https://github.com/pushchain/push-chain-core-contracts/tree/15b70544feba1586f488a90e38650bf33ea3c634/docs</a> |

| Asset                                                                                     | Type           |
|-------------------------------------------------------------------------------------------|----------------|
| CEA.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]          | Smart Contract |
| CEAFactory.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]   | Smart Contract |
| CEAMigration.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main] | Smart Contract |
| CEAProxy.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]     | Smart Contract |
| Errors.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]       | Smart Contract |
| PRC20.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]        | Smart Contract |
| Types.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]        | Smart Contract |
| UEA_EVM.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]      | Smart Contract |
| UEA_SVM.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]      | Smart Contract |
| UEAFactory.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]   | Smart Contract |
| UEAMigration.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main] | Smart Contract |
| UEAProxy.sol [https://github.com/pushchain/push-chain-core-contracts/tree/audit-main]     | Smart Contract |

| Asset                                                                                                                                                                             | Type           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| UniversalCore.sol [ <a href="https://github.com/pushchain/push-chain-core-contracts/tree/audit-main">https://github.com/pushchain/push-chain-core-contracts/tree/audit-main</a> ] | Smart Contract |
| Utils.sol [ <a href="https://github.com/pushchain/push-chain-core-contracts/tree/audit-main">https://github.com/pushchain/push-chain-core-contracts/tree/audit-main</a> ]         | Smart Contract |
| WPC.sol [ <a href="https://github.com/pushchain/push-chain-core-contracts/tree/audit-main">https://github.com/pushchain/push-chain-core-contracts/tree/audit-main</a> ]           | Smart Contract |

## Appendix 3. Additional Valuables

### Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

### Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.